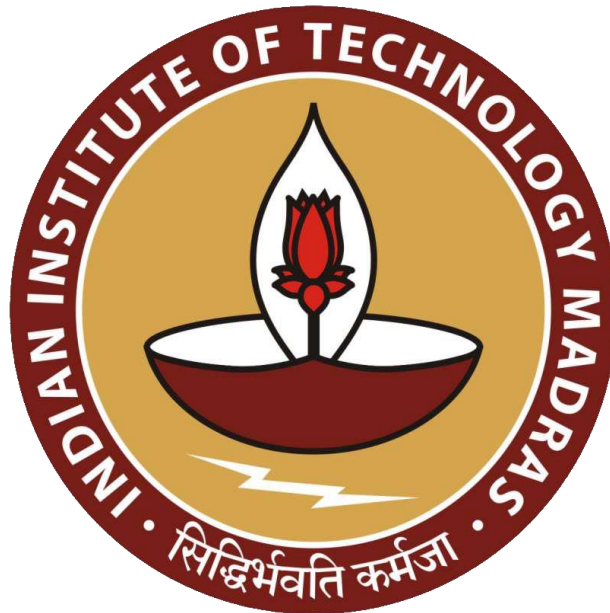


Software Engineering Project

Jan 2025 Term - Team 36

Milestone 6



Team Details

Sri Harsha Reddy Sagili

(23f2005056)

Sukrit Gupta (21f1000096)

Sharma Priyanshu Dilip(21f2001068)

Subhojit Kumar

Patar(21f1003610)

Ritwik Ghosh(21f2001150)

Sameep Arun

Kambli(21f2001228)

Vaibhav Gupta(21f2001529)

28.03.2025

IIT Madras BS Degree Program

Contents

- 1. Milestone 13**
- 2. Milestone 2.....5**
- 3. Milestone 3.....6**
- 4. Milestone 4.....19**
- 5. Milestone 5.....34**
- 6. Milestone 6.....46**

Milestone 1

1. IDENTIFYING TYPES OF USERS

For the seek portal we have identified 3 user divisions as per requirements. They are as follows:

1. **Primary Users:** The students pursuing the degree will have a higher interaction with the AI agent and will use it for self learning purposes and better performances in their course.
2. **Secondary Users:** The secondary users will be the professors and the teaching staff, who will enforce a specific direction to the AI agent and will upload relevant course material for the AI agent to train from. Their essential concern will be to ensure that no malpractices or unethical use of the AI agent is propagated in the self learning tool.
3. **Tertiary Users:** The tertiary users will be the IT Staff or any administrators of the course that will be helping in the maintenance of the platform for a better experience for the students.

2. User Stories

1. Primary Users

1. As a student,

I want to plan my weekly study schedule, where the AI agent recommends alternative resources, categorizes topics by difficulty, and provides a structured study plan based on a standardized framework.

2. As a student,

I want the AI agent to suggest practice problems based on my performance in practise assignments,
So that I can improve my skills in weak areas.

3. As a student,

I want to receive tips for improving my programming and debugging skills based on my submission on programming assignments,
So that I can perform better in coding assignments and quizzes.

4. As a student,

I want the AI agent to summarize key concepts from videos or resources I've already studied in my mother tongue,
So that I can quickly grasp the key points in a language I understand.

Milestone 1

5. **As a student,**

I want the AI agent to offer alternative explanations for complex topics, So that I can understand the subject matter more comprehensively.

6. **As a student,**

I want the AI agent to suggest additional external learning resources (e.g., online articles, tutorials, or videos) when I ask any question regarding to the concept mentioned in the course week,

So that I can explore a topic further outside the course material.

7. **As a student,**

I want the AI agent to sustain a conversation while responding in relation to the conversation history.

So that I can get my answers with respect to the context and continuity of our previous interactions.

8. **As a student,**

I want to be able to create new support/query tickets

So that I can get help with my issues from the support team.

2. Secondary Users

1. **As a professor,**

I want the system to evaluate the assignments submitted by students, So that I can save time and focus on reviewing the key aspects of their work.

2. **As a professor,**

I want the system to identify pain points or common mistakes from students' submissions, So that I can address those areas in my teaching or provide targeted feedback.

3. Tertiary Users

1. **As a member of the IT support/admin team,**

I want the system to evaluate GitHub functionality to track project completion based on milestone submission counts,

So that I can monitor students' progress and ensure timely project completion.

2. **As a member of the IT support/admin team,**

I want the AI agent to analyze user-reported bugs and suggest probable causes within the codebase.

Milestone 2

PPT SLIDE FOR MILESTONE 2

https://drive.google.com/file/d/1OrGBX_F_Ua7H1rWgNGMokdprPBfduqC4/view?usp=sharing

USER STORIES VEDIO:

https://drive.google.com/file/d/1nifwlEcv9G1Y-yQ-HvGSBfNvRFfiyZp/view?usp=drive_link

Milestone 3

Project Schedule

Complete Schedule

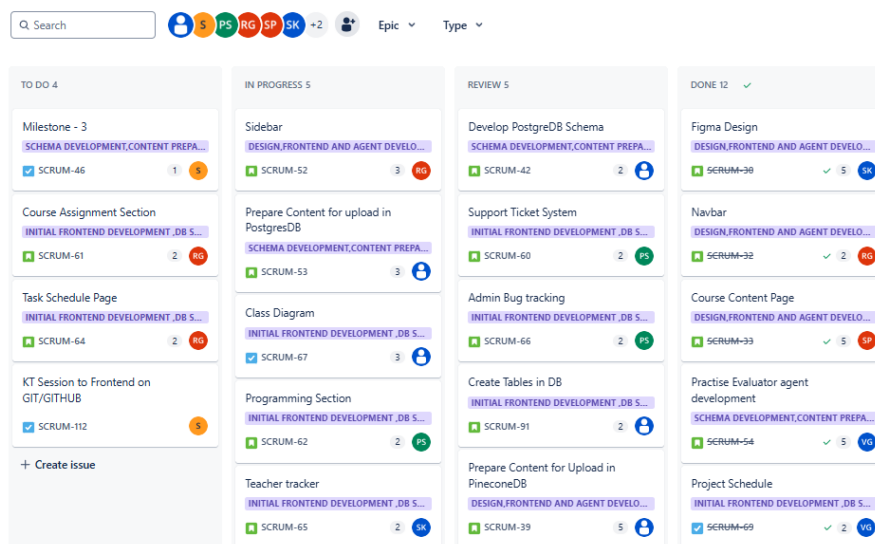
The whole project is divided into different software development lifecycle phases consisting of Planning, Designing, Development Testing and Maintenance. Also we planned these different phase in alignment with the Milestones. The following is the complete schedule of our project:

Sprint	EPIC	Start Date	End Date	Goals
1.	Foundation Setup for AI Agent Development	22/01/2025	03/02/2025	1. RAG Exploration 2. Milestone 1 3. Identify user stories 4. Design Exploration
2.	Design, Frontend and Agent Development	04/02/2025	10/02/2025	1. Design development 2. Common frontend component development 3. Content preparation 4. Agent development 5. Code Infrastructure setup 6. Milestone - 2
3.	Schema Development, Content Preparation and Agent Development	11/02/2025	21/02/2025	1. Schema development 2. API development 3. Content preparation 4. Agent development 5. Milestone – 3 6. Relevant search development
4.	Initial Frontend Development, DB Setup and Agent Development	11/02/2025	21/02/2025	1. Table creation 2. Frontend page development 3. Agent development
5.	Enhance Frontend, Prepare DB Content and Agent Development	22/02/2025	27/02/2025	1. Frontend enhancement 2. Upload content on DB 3. Agent development
6.	Backend Integration and Agent Development	20/02/2025	05/03/2025	1. Backend integration 2. Authentication integration 3. Enhance agent 4. Milestone - 4
7.	Testing and Integration	06/03/2025	21/03/2025	1. Testing 2. Backend Integration 3. Agent development 4. Milestone - 5
8.	Hosting	22/03/2025	03/04/2025	1. Host frontend 2. Host backend 3. Milestone – 6 4. Containerization

Scheduling tool and Scrum Board:

The tool which we used for project management in JIRA. Following image is a screenshot of the JIRA scrum board for the ongoing sprint 4.

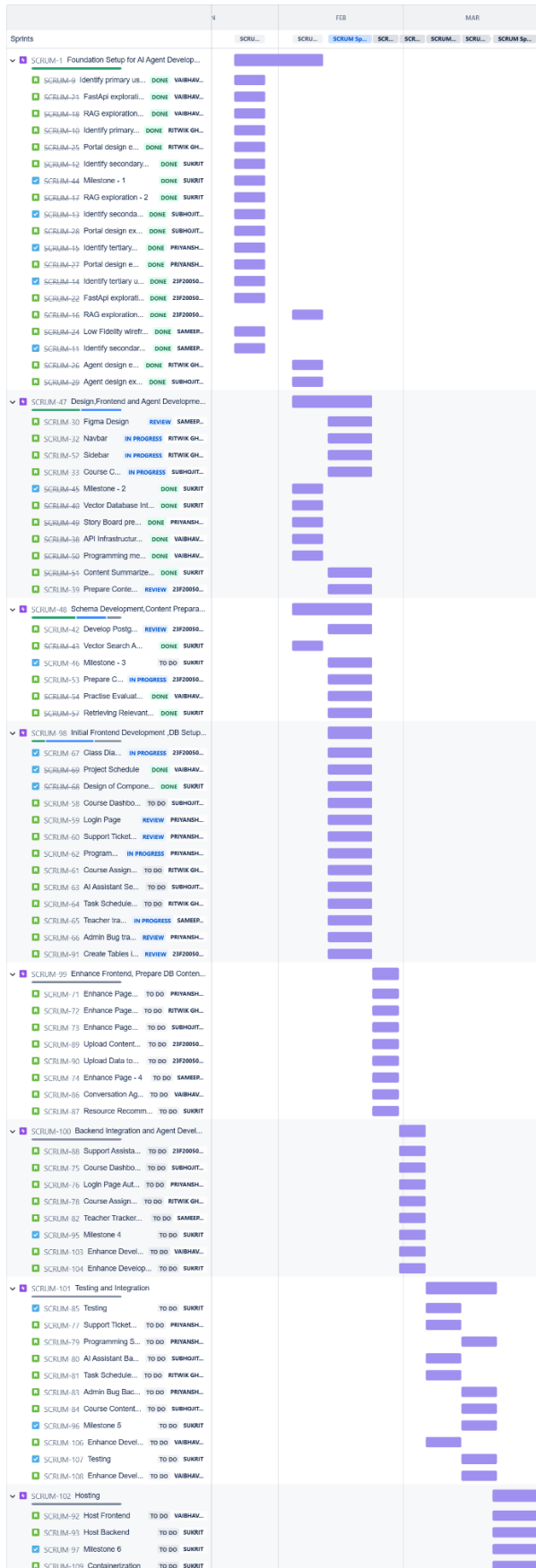
SCRUM Sprint 4



Milestone 3

Detailed schedule:

The Detailed schedule of our project including the subtask for each sprint is as follows:



SS

Milestone 3

Scrum Meeting

Scrum standup Schedule

Scrum standups are scheduled to track progress, assign new tasks, and incorporate new requirements in an agile workflow. Our team follows this schedule:

- **Every Monday:** Team standup (30 min)
- **Every Wednesday:** Client meeting (30 min) + Team standup (30 min)

Minutes of Meeting

Details of few of the scrum meeting are provided below. Each Scrum meeting explains the minutes of the meeting.

1. Scrum 1 (22 Jan, Duration – 1 hours)
 - a. Attendance – Full
 - b. Points discussed – project planning, discussion regarding topic of interest for each team member, further bifurcation of team into Backend and Frontend based on their areas of interest, and assignment of user stories to team members.
 - c. Short description - Team discussed the exploration needed to achieve the project. Pre requisite, resources, time constraint, interest of each team member was discussed. User stories of different user types got assigned to respective team members. Design complexities were also discussed during this meeting
 - d. Plan of action – each team member was required to explore the project, in order to get a better understanding of the project and milestone requirements
2. Scrum 2 (27 Jan, Duration – 1 hour)
 - a. Attendance – Full
 - b. Points discussed – Update on user stories, finalization of milestone 1, and frontend design.
 - c. Short description - Team discussed the progress of their currently assigned stories. Finalization of the user stories. Team also discussed on the design of different pages based on user stories requirements.
 - d. Plan of action – Frontend team tasked with the project Design, Backend tasked with designing the basic project structure and exploration of various tools to be used.
3. Scrum 3 (31 Jan, Duration – 1 hour)
 - a. Attendance – everyone expect Harsha has Joined
 - b. Points discussed – Further design discussion & design Finalization, Database discussion, and RAG discussion.
 - c. Short description - Team discussed the outcomes of the exploration did on RAG implementation. Discussion included Db to be used, embedding, similarity search, Multi agent architecture. Team also discussed the Tech Stack they are comfortable with development. Final draft of low fidelity design was discussed.
 - d. Plan of action – make necessary adjustments to the design, initializing the database in tools like Vercel, Pinecone, etc. and uploading course content in the db.
4. Scrum 4 (7 Feb, Duration – 1 hour)
 - a. Attendance – everyone except Sameep has Joined.

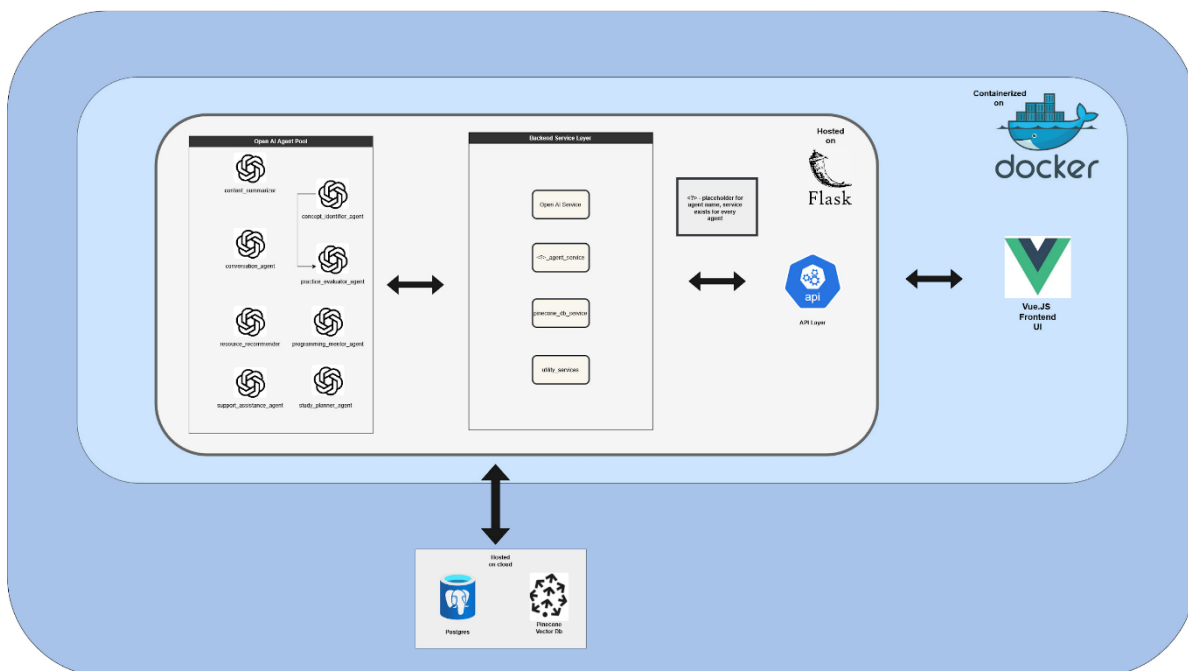
Milestone 3

- b. Points discussed – Assigned frontend pages, checked team members' status, delegated the database schema task, and allocated API endpoint development.
- c. Short description – team discussed regarding milestone 3 and delegated the tasks to team members based on milestone 3 requirements. We have also gone beyond milestone 3 and started working on milestone 4 requirements.
- d. Plan of action – developing all the frontend pages, developing db scheme, class diagrams and ER diagrams, creating few end points.

Software Component Design:

We have divided our components into backend and frontend and the component design for each of them are as follows.

Backend Component:



Backend Application Description:

The application is built using Fast API and integrates PostgreSQL DB hosted on Vercel. It implements multiple RAG agents the leverage Pinecone Services. These agents interact with the DB and other utilities through a service layer, which provides various functionalities to support their operation.

Agent Description

Milestone 3

1. Study Planner Agent

- Function: Retrieves weekly study content from Pinecone and organizes a structured study schedule based on the user's availability and preferences. It gathers input through a form and ensures an optimal balance of study sessions.
- Primary User Story: As a student, I want to plan my weekly study schedule, so I can efficiently manage my time and cover all necessary topics. This agent retrieves weekly content from Pinecone and generates a structured study plan based on user inputs through a form.

2. Practice Evaluator Agent

- Function: Analyzes the user's submitted assignments, identifies mistakes, and suggests additional practice questions tailored to weak areas for improvement.
- Secondary User Story: As a student, I want to evaluate my assignment performance and receive extra questions for practice, so I can improve in areas where I struggle. This agent assesses mistakes in assignments and suggests additional practice questions for better understanding.

3. Programming Mentor Agent

- Function: Reviews working code, identifies inefficiencies, and provides suggestions for optimization, improving structure, performance, and maintainability.
- Primary User Story: As a programmer, I want to receive suggestions for optimizing my working code, so I can improve efficiency and structure. This agent analyzes the code, identifies areas of improvement, and suggests optimizations.

4. Concept Identifier Agent

- Function: Examines mistakes in assignments or practice problems, extracts the underlying concepts, and helps users strengthen their foundational understanding.
- Primary User Story: As a learner, I want to understand the core concepts behind my mistakes, so I can strengthen my foundation in the subject. This agent extracts key concepts from questions and answers, helping users identify gaps in their knowledge.

5. Content Summarizer Agent

- Function: Retrieves lecture content from a database, transcribes, summarizes key concepts, and translates them into the student's preferred language for better comprehension.
- Primary User Story: As a student, I want to receive key lecture concepts in my mother tongue, so I can understand them more clearly. This agent retrieves lecture content from the database, transcribes, and translates it into the user's preferred language.

Milestone 3

6. Conversation Agent

- Function: Maintains contextual conversations by extracting conversation history from the database and retrieving relevant data from Pinecone, ensuring continuity and relevance in responses.
- Primary User Story: As a user, I want to have a meaningful conversation that considers previous interactions, so I can receive relevant responses. This agent extracts conversation history and related data from Pinecone to maintain contextually aware discussions.

7. Resource Recommender Agent

- Function: Fetches relevant learning resources such as articles, tutorials, or videos from Pinecone based on user queries and study requirements.
- Primary User Story: As a learner, I want to receive additional external learning resources, so I can explore topics further. This agent retrieves relevant data from Pinecone and suggests articles, tutorials, or videos based on the user's needs.

8. Support Assistant Agent

- Function: Analyzes user-reported bugs, reviews the codebase, and identifies potential problem areas to suggest probable causes and debugging solutions.
- Tertiary User Story: As a developer, I want to analyze reported bugs and identify probable causes, so I can debug efficiently. This agent examines user-reported bugs, inspects the codebase, and highlights areas prone to issues.

The frontend of the learning application will provide an intuitive and seamless user experience for students, instructors, and administrators. It consists of various components designed to facilitate learning, assessments, and administrative functions

Navbar Component

- A horizontal navigation bar at the top.
- Includes a logo for branding and redirection.
- Displays user roles for contextual access.
- Provides a secure log-in/sign-out option.

Sidebar Component

- A vertical navigation menu for structured access based on user roles.
- Features a search bar for quick navigation.
- Includes icons for home, support, notifications, settings, and profile.

Milestone 3

SS

Login Page

- Secure credential input fields for authentication.
- Google sign-in option for ease of access.
- Motivational content or branding elements.

Student Dashboard (Seek Portal)

- Serves as a central hub for student activities.
- Provides access to 3.1.5 (Course Assignment Section), 3.1.6 (Coding Practice Section), and 3.1.7 (Course Content Page).
- Displays enrolled courses and key performance metrics.
- Includes CGPA overview and course details.
- Navigation sidebar for profile, settings, and support.

Course Assignment Section

- Week-based navigation for structured assignments.
- Displays MCQs and practice assignments.
- Includes submission and practice buttons.

Coding Practice Section

- Provides an interactive coding environment.
- Includes a functional code editor with test and submit options.
- AI-generated tips for assistance.

Course Content Page

- Features a video player for lecture videos.
- Summarization feature for quick review.
- Explanations panel in default and native languages.

Support Ticket System

- Allows users to submit support tickets.
- Categorization for efficient issue resolution.
- File attachment and submission options.

AI Assistant (Main Chat, My Space, Schedule Plan Sections)

- Interactive AI assistant for queries and suggestions.
- Customizable chat and predefined conversation options.
- Personal workspaces for study material organization.
- Task planning with AI-suggested schedules.

Milestone 3

Teacher Dashboard

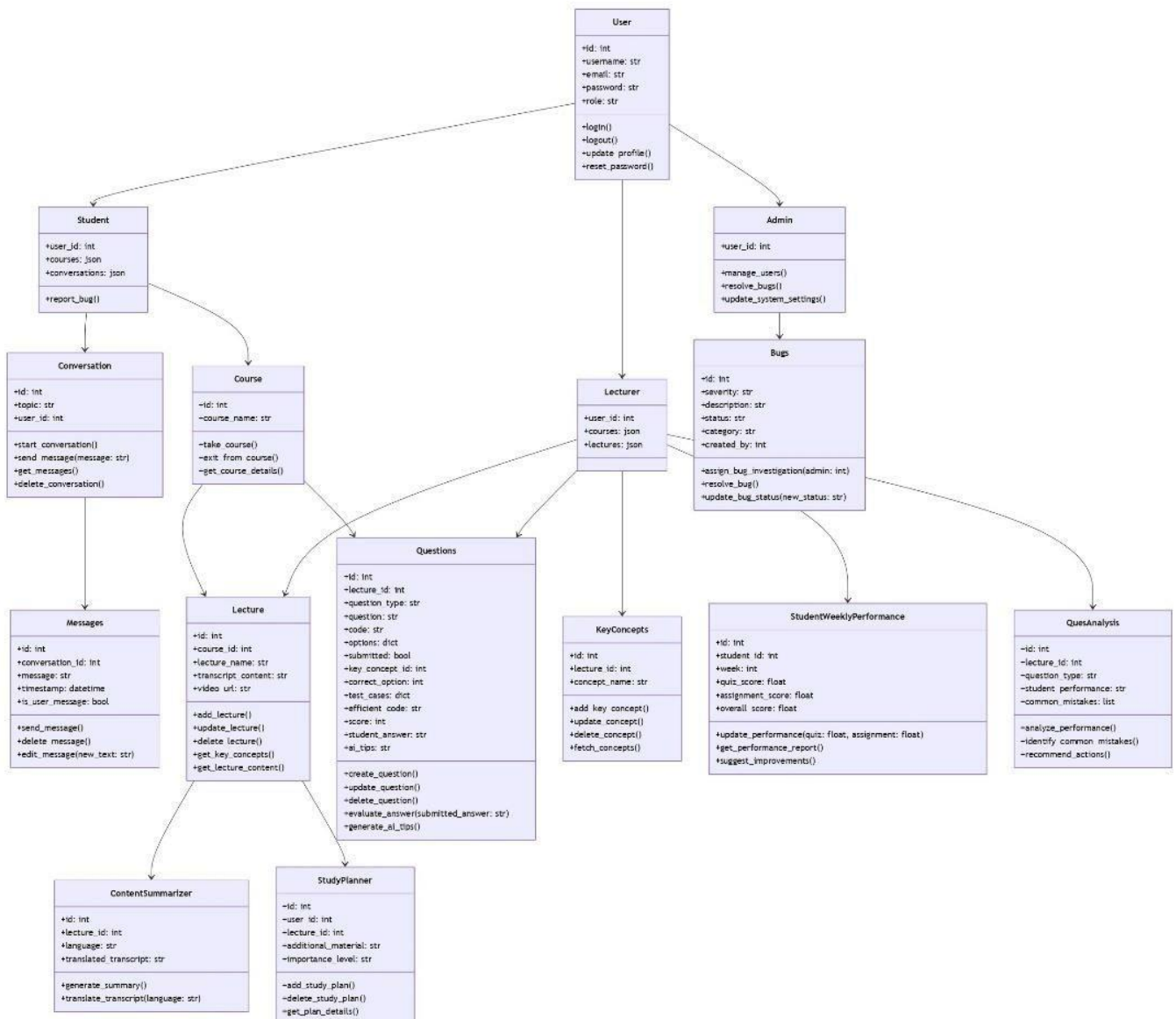
- Assignment review section with filters and pagination.
- Student performance tracking with analytics.
- Course management tools for content upload and AI settings.

Admin Dashboard

- GitHub activity tracking for coding progress monitoring.
- Milestone tracking for student projects.
- Summarized metrics and activity graphs.

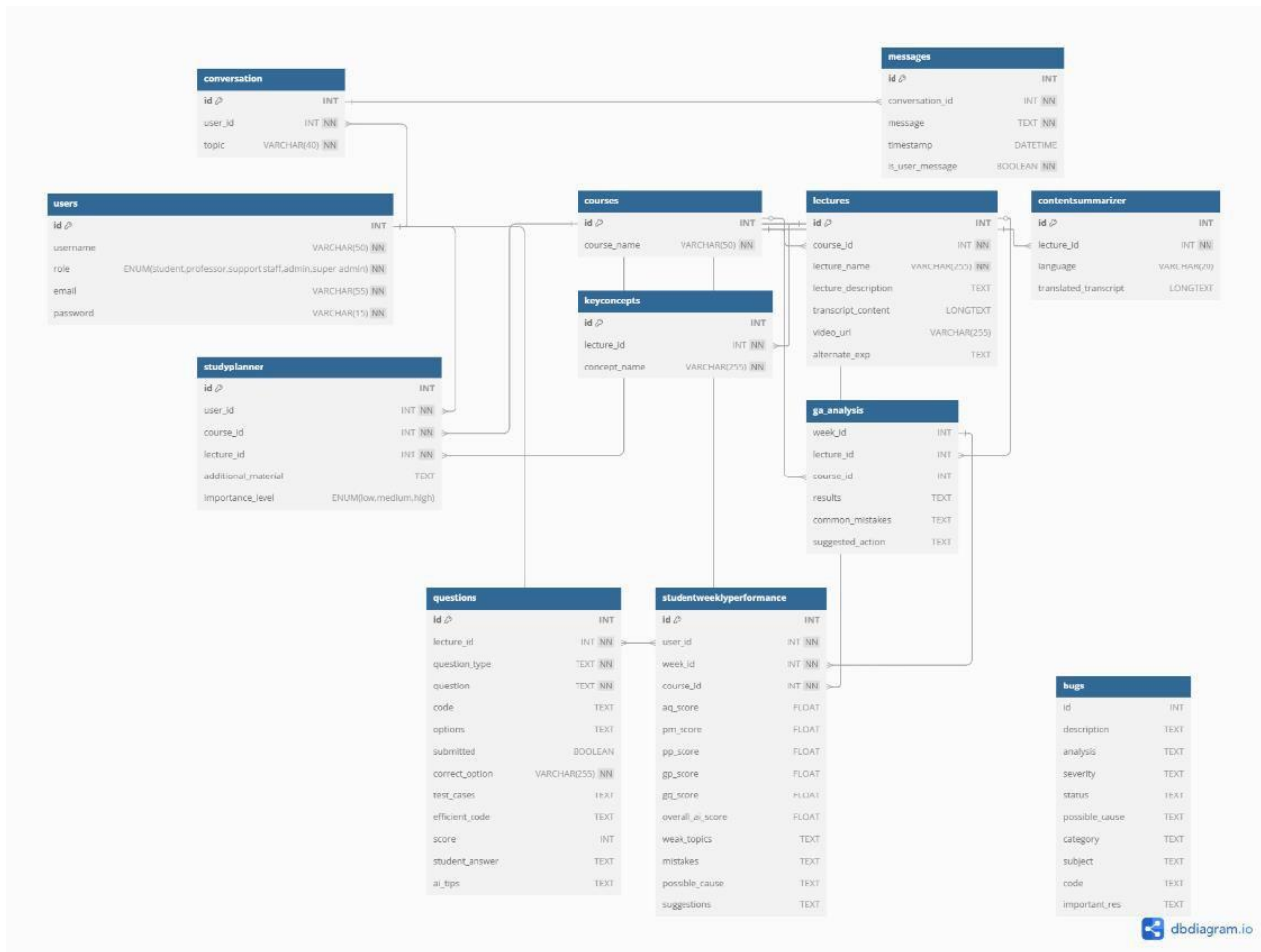
The frontend is developed using Vue.js for an enhanced user experience. The integration of AI tools further enhances user engagement and learning outcomes.

Class diagram



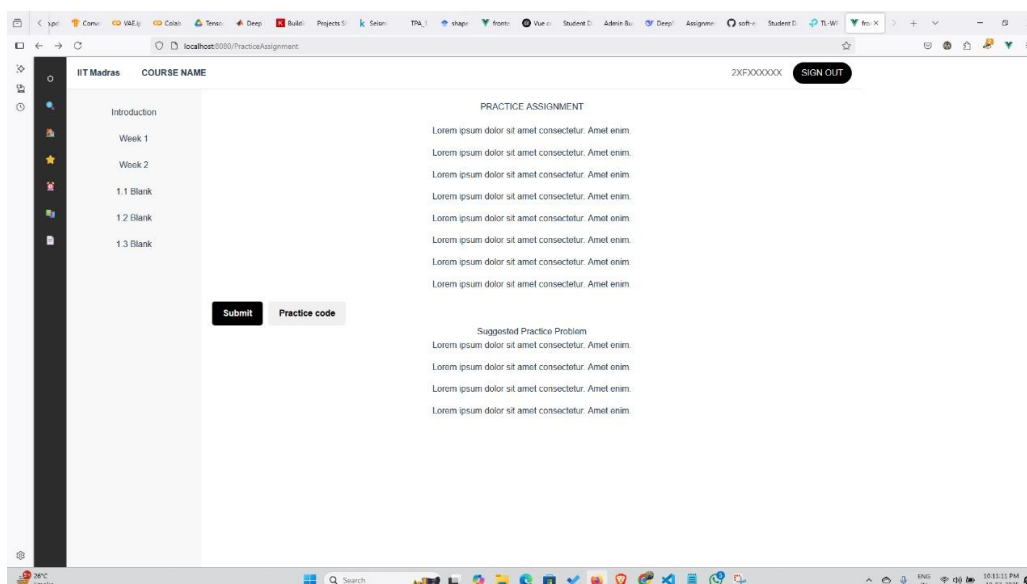
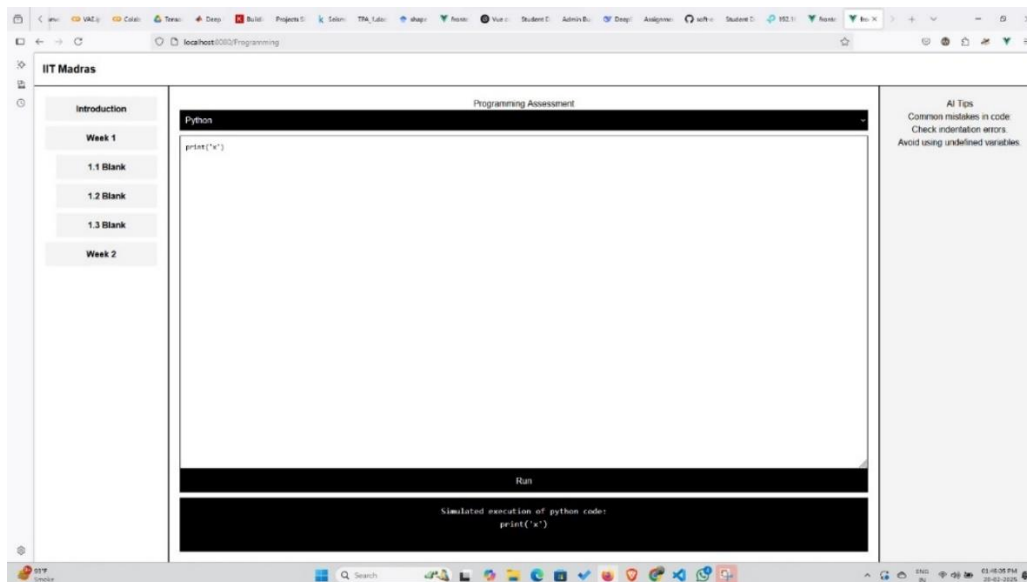
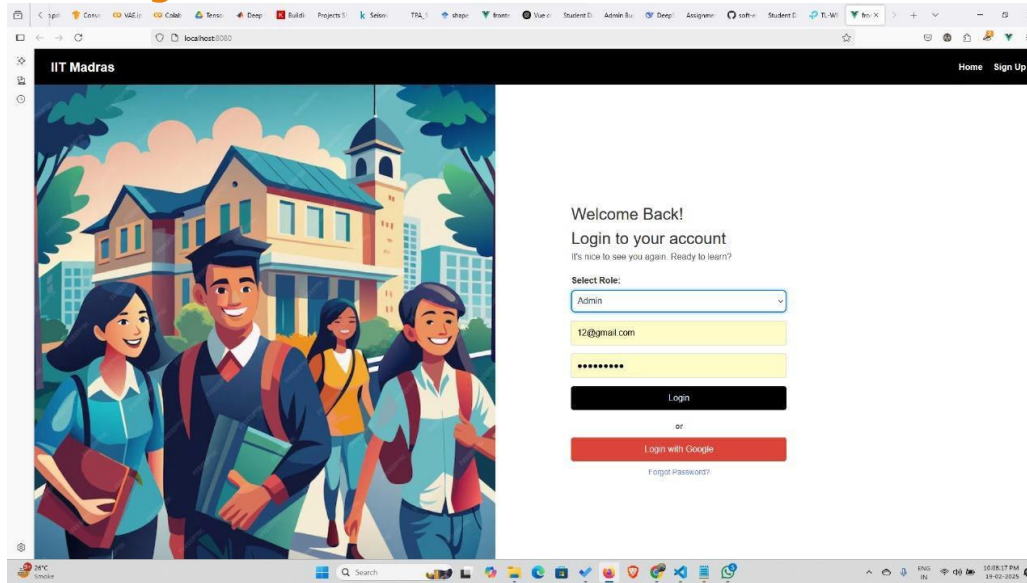
Milestone 3

ER diagram:

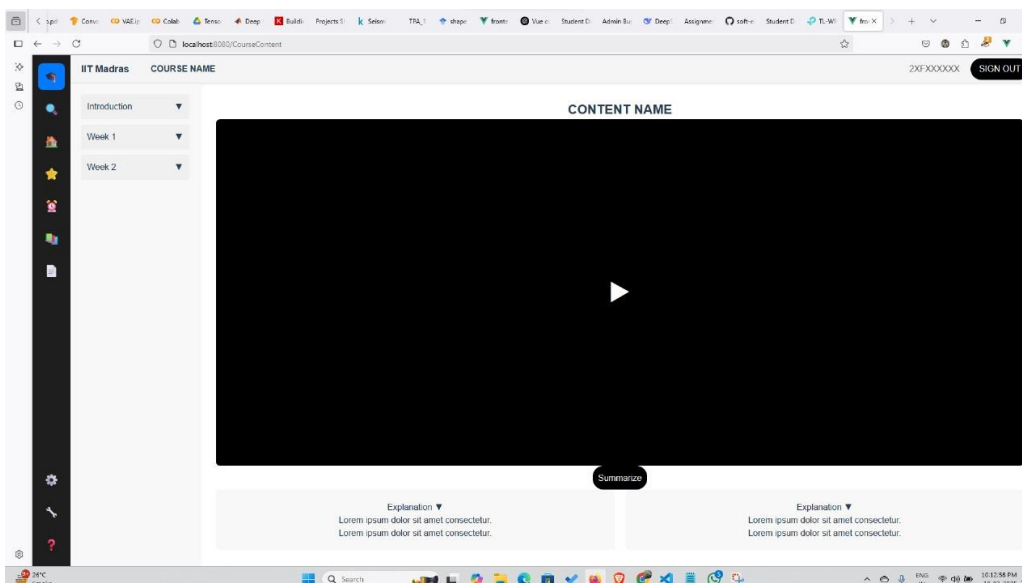
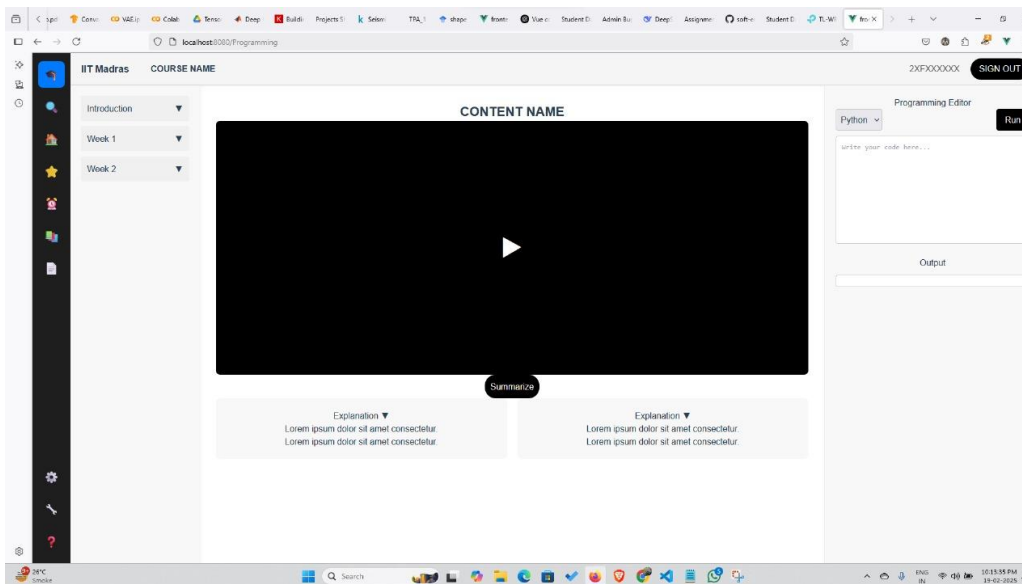
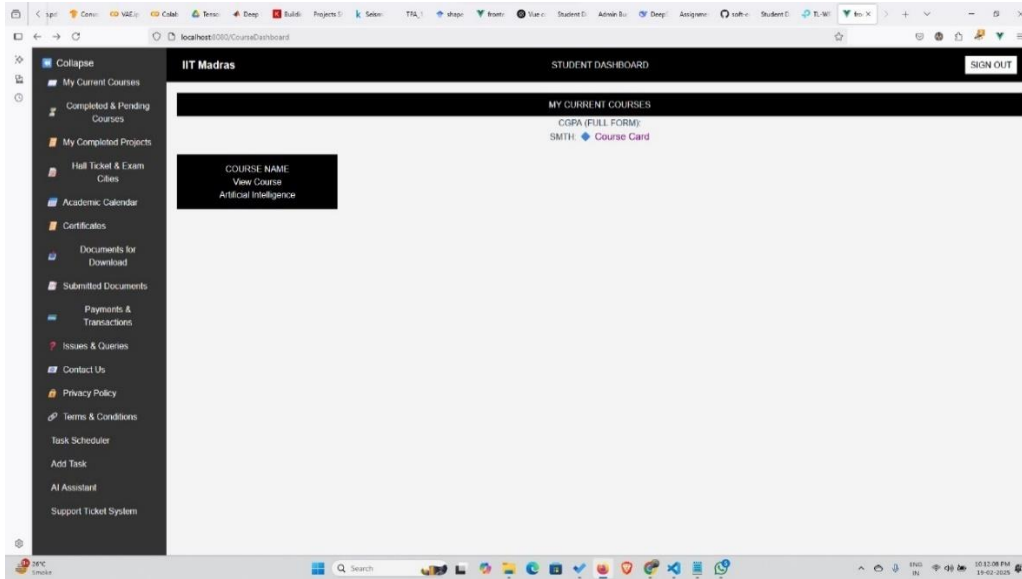


Milestone 3

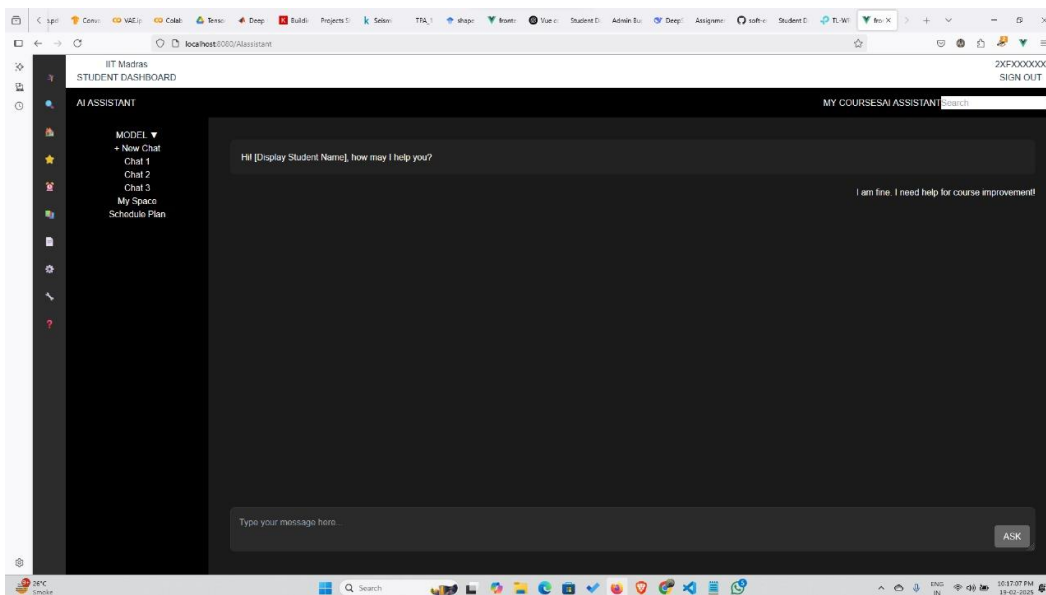
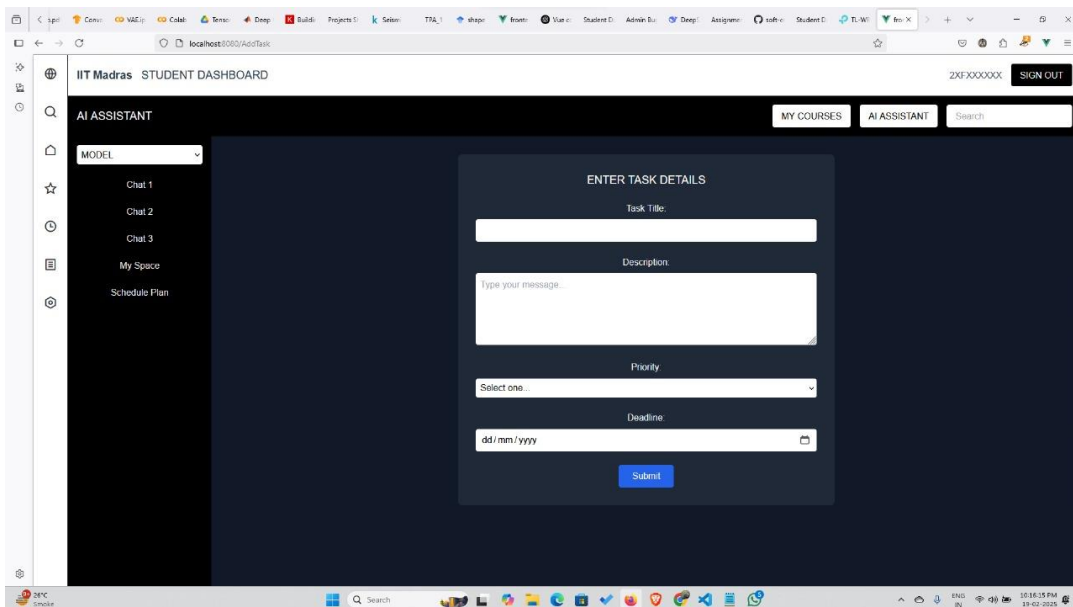
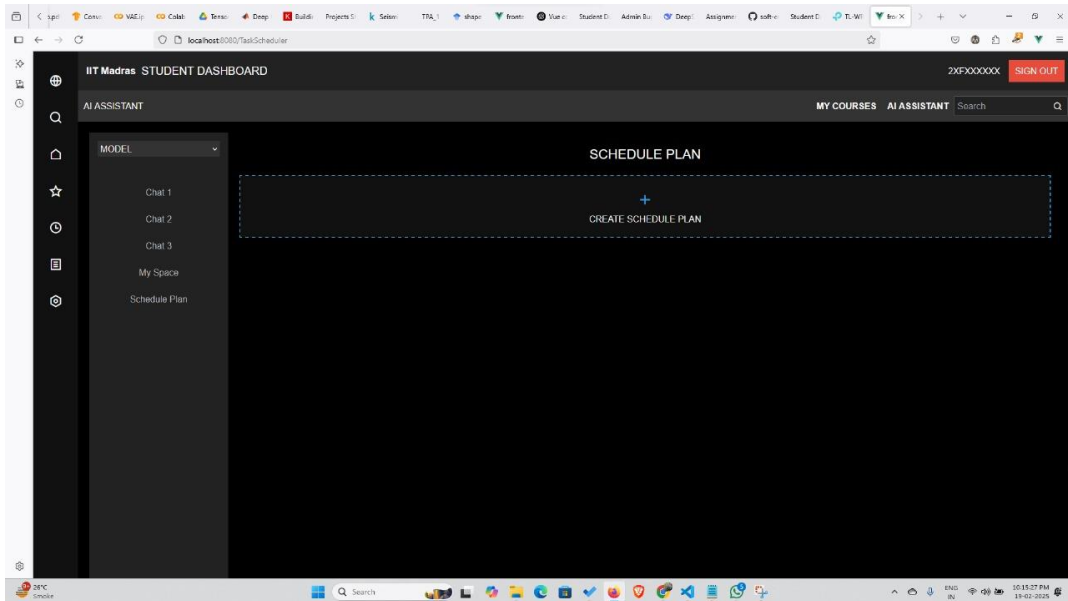
VUE Pages



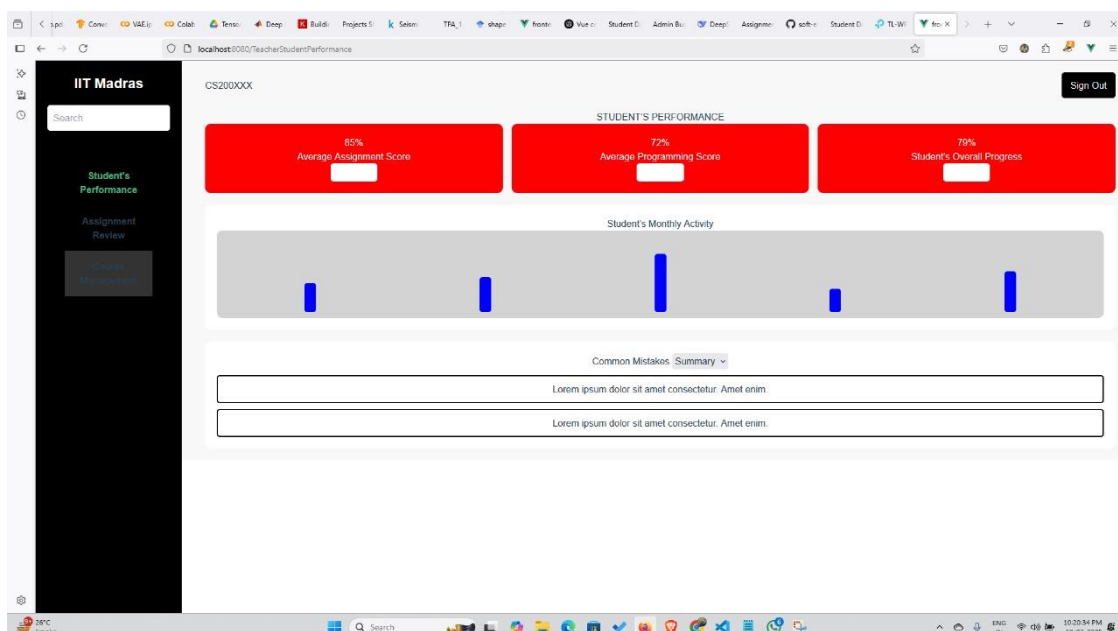
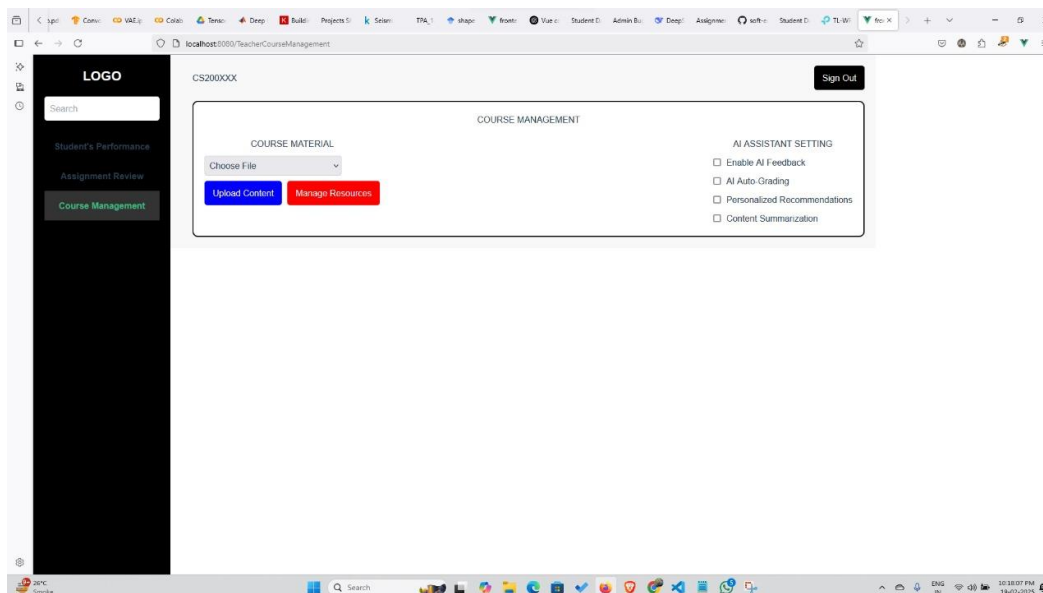
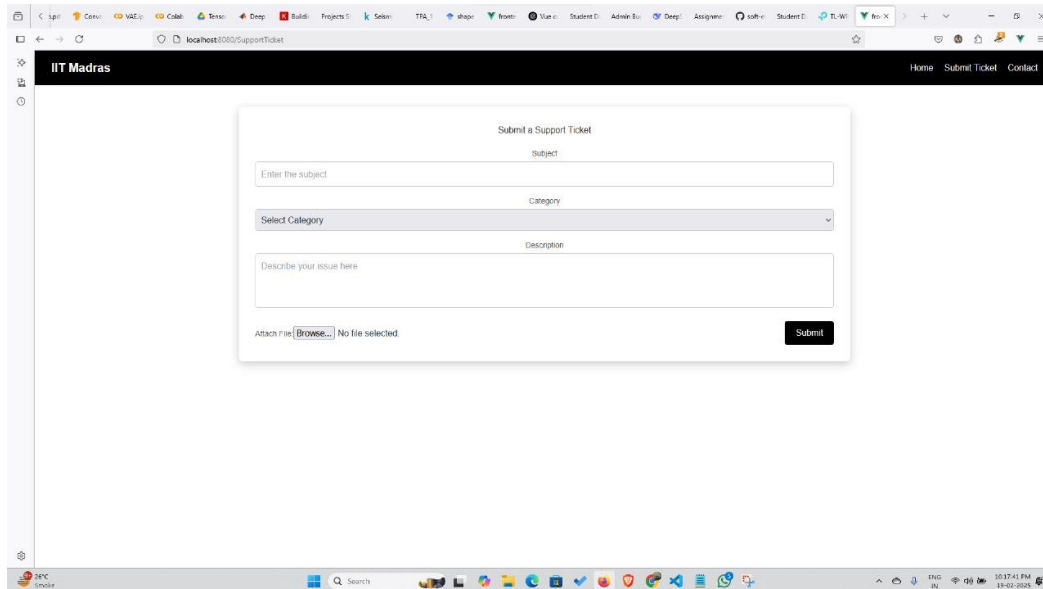
Milestone 3



Milestone 3



Milestone 3



Milestone 4

1. List of APIs Integrated

S.No.	API	Purpose
1	Pinecone API	Knowledge retrieval from vector database
2	OpenAI GPT API	Language model inference for generating responses
3	LangChain	Orchestration and query processing
4	FastAPI	API framework for serving endpoints

2. List of APIs Created

S.No.	API	Endpoint	Method	Purpose
1.	Programming Mentor Agent	/api/programming-mentor	POST	Analyzes student's working code and suggests optimizations
2.	Practice Evaluator Agent	/api/practice-evaluator	POST	Identifies mistakes in assignments, extracts core concepts, and generates extra practice questions
3.	Conversation Agent	/api/conversation	POST	Engages in dynamic conversation and retrieves relevant knowledge from Pinecone
4.	Resource Recommender Agent	/api/resource-recommender	POST	Retrieves resource from pinecone needed for the query asked by the student.

Milestone 4

5.	Mother Tongue Summarizer Agent	/api/{route}		Summarize the video lectures in mother tongue of the student
6.	Study Planner Agent	/api/{route}	POST	Suggest a well-defined study plan to the student according to their available time, difficulty and priority
7.	Support Assistance Agent	/api/{route}	POST	Suggest critical code block which may require to look upon.
8.	Login	/api/verify-user	POST	Verify existing user and log it in the application
9.	Create Lecture	/api/create-lecture	POST	Route to create lecture in the DB
10.	Fetch Lecture	/api/get-lecture/week/{week_id}	GET	Fetch all lecture in that particular week
11.	Practice Assignment Question	/api/mcq-questions/week/{week_id}	GET	Fetch mcq question in that particular week
12.	Coding Assignment Question	/api/coding_question/week/{week_id}	GET	Fetch coding question with it's test cases in that particular week
13.	Courses	/api/courses/{user_id}	GET	Fetch courses taken up by a user
14.	Bug Reporting	/api/report-bug	POST	Report a new bug
15.	Get Bugs	/api/get-bugs	GET	Get all bugs
16.	Get Bug	/api/get-bug/{bug_id}	GET	Get a specific bug
17.	OAuth Login		POST	
18.	Create User	/api/create-user	POST	Create new user in the DB

Milestone 4

3. Description of API Endpoints

a. Programming Mentor Agent

Request:

```
{
  "coding_question": "Write a Python function to check if a number is prime."
  "student_code": "def is_prime(n): return all(n % i != 0 for i in range(2, n))"
}
```

Response:

```
{
  "optimization_feedback": "***Tips for Improvement:**\n- Handle edge cases for numbers less than 2.\n- Limit the loop to check divisibility up to the square root of `n` for efficiency.\n- Add docstrings to explain function behavior.\n\n**Optimized Code:**\n```\npython\nimport math\n\ndef is_prime(n):\n    \"\"\"Check if a number is prime.\"\"\"\n    if n < 2:\n        return False\n    for i in range(2,\n        int(math.sqrt(n)) + 1):\n        if n % i == 0:\n            return False\n    return True\n```\n"}
}
```

b. Practice Evaluator Agent

Request:

```
{
  "submissions": [
    {
      "assignment_question": "Explain the concept of recursion with an example.",
      "correct_answer": "Recursion is a process where a function calls itself.\nExample:\n```\npython\ndef factorial(n): return 1 if n == 0 else n * factorial(n-1)\n```\n",
      "student_answer": "Recursion is when a function repeats its operations until a condition is met."
    },
  ],
}
```

Response:

```
{
  "evaluations": [
    {
      "assignment_question": "Explain the concept of recursion with an example.",
      "core_concept": "Core Concept: Understanding Recursion",
      "feedback": "***Mistakes Identified:**\n- Your definition of recursion is somewhat vague; it doesn't clearly explain that recursion involves a function calling itself.\n- The explanation lacks an example, which is crucial for illustrating the concept of recursion.\n- The phrase \"repeats its operations\" could be misleading, as it may imply iteration rather than self-calling.\n\n**Extra Practice Questions:**\n1. Define recursion and explain the base case in a recursive function.\n2. Write a recursive function to calculate the Fibonacci sequence.\n\n**Correct Answers for Practice Questions:**\n1. Recursion is a process where a function calls itself to solve a problem. The base case is a condition that stops the recursion, preventing infinite calls. For example, in a factorial function, the base case is when n equals 0, where the function returns 1.\n2. Here's a simple recursive function to calculate the Fibonacci sequence:\n```\npython\ndef fibonacci(n):\n    return n if n <= 1 else fibonacci(n-1) + fibonacci(n-2)\n```\n"}
  ],
}
```

Milestone 4

c. Conversation Agent

Request:

```
{  
  "user_id": 1,  
  "message": "what is the last question which i asked you?"  
}
```

Response:

```
{  
  "response": "The last question you asked me was, \"what is the full form of DSA.\" If you  
  have any more questions or need further information, feel free to ask!"  
}
```

d. Resource Recommender Agent

Request:

```
{  
  "query": "Tell me about N-ary Tree"  
}
```

Response:

```
{  
  "result": "To understand N-ary Trees, refer to the following pages from the book Algorithm  
  Design by Jon Kleinberg:  
  \n\n1. Page 78 (ID: 104.1): Introduces rooted trees as  
  fundamental objects in computer science, discussing their hierarchical structure and  
  applications, including examples of organizational structures.  
  \n\n2. Page 78 (ID: 104.3):  
  Continues the discussion on trees, proving important facts about n-node trees and their  
  properties, such as the relationship between nodes and edges.  
  \n\nThese sections provide  
  valuable insights into the characteristics and significance of N-ary Trees in computer science."  
}
```

e. Login

Request:

```
{  
  "email": "vaibhav123@gmail.com",  
  "password": "securepassword"  
}
```

Response

Milestone 4

```
{
  "message": "User verified",
  "user_id": 1
}
```

f. Create Lecture

Request:

```
{
  "week_id": 1,
  "lecture_name": "Introduction to AI",
  "transcript_content": "Welcome to the AI lecture...",
  "video_url": "https://example.com/video.mp4"
}
```

Response:

```
{
  "week_id": 1,
  "lecture_name": "Introduction to AI",
  "transcript_content": "Welcome to the AI lecture...",
  "video_url": "https://example.com/video.mp4",
  "id": 17
}
```

g. Fetch Lecture

Response:

```
[ {
  "week_id": 5,
  "lecture_name": "Introduction to pdsa",
  "transcript_content": "Programming Data Structures and Algorithms using Python Professor Madhavan Mukund \nJupyter Notebooks
\nSo, welcome to the course Programming Data Structures and Algorithms using Python. So, to begin with we will look at some of the things
that you are already familiar with just to get a refresher on Python and one of the things I will start with is the environment in which we
are going to run Python code. So, I am going to be talking to you today about Jupyter notebooks. \n(Refer Slide Time: 0:32) \nSo, normally when
we want to write and run code there are many options available to us, perhaps the simplest option is what you see here, so you have a text
editor, so you type your code and then you open a kind of a console or a terminal and then you load the code that you have typed in in your text
editor and then you run it. \nNow, this is of course, easy to do but it is a little bit tedious because if you decide to make a change over here then
you have to go back, reload it, so going around in this kind of cycle of edit and run is a little bit cumbersome if you are using a separate text
editor and using directly the console. \n(Refer Slide Time: 1:16) \nSo, this gives rise to a more convenient interface, which you are familiar with
REPLIT called an Integrated Development Environment or IDE. So, in an IDE you have on one side something where you can edit your code
and side by side you have this command window where you can run the code and see the output. \nAnd because you can now see the output and
edit your code in the same interface it is much easier to cycle back and forth and see how your code can be changed and how the output
changes. So, you have this quick cycle of updating your code and running it, and of course, an IDE has other features also, so it offers you
typically a debugger. \nA debugger will allow you to interrupt your code and inspect the values of various variables to see what is going on.
You might also be able to prepare some test cases in advance and run them to see how your code performs and so on. So, this is the default that
most people use for developing serious amounts of code, and of course different IDEs are designed for different programming languages.
\nSome like REPLIT support multiple languages; other IDEs you might find are dedicated to certain languages like Python or Java or C plus
plus and so on. So, we already have an IDE but I am going to present to you yet another way of writing and running code and the question is
why would one want one more when you already have an IDE. \n(Refer Slide Time: 2:48) \nSo the main point that I would like to emphasize is
that writing code whether for yourself or for teaching, like I am teaching now, also involves collaboration, very rarely do you write code only
for yourself, you usually work in a team. So, as a team you might be together developing the code or this might be a larger team. \nFor
instance, it could be some kind of a research environment and this is very common, say in machine learning, you are trying to solve a problem,
coding is part of the solution but is not the solution in itself. So, when you run the code not only it is possible that more than one person may be
running the code or developing the code to run. \nBut there are maybe people who are not running the code, who are not even aware of how the
code works but who might want to know what the code does. So, then you want to be able to share your results. Now, not notice that when you
have a typical IDE, when you run the code you can see the results, but what happens after you run the code. You cannot take the code, you
cannot take the IDE open to another office and show somebody what you have done. \nYou would like to have some way of preserving the
output for later, without having to go through a sort of cumbersome process of saving it to a file or preparing a report or something like that.
So, here on the right you see a typical view of what a Jupyter notebook, which is an interface that I am going to introduce today looks like. \nSo,
you have on one side the ability to write text, so in this case is just a simple title, but you can also, as we will see, introduce more annotations
which describe what you have done or what you are planning to do and then you have in this block, you have the actual code exactly \nas you
would have typed it into a standard editor or into an IDE and now at the bottom you have outside this grey area, you have the output of the
code what you would normally see in a separate window on the side in an IDE. \nSo, the documentation is interleaved with the code, so you
have the documentation here, you can have more documentation, so in between your code fragments or different parts of your code, you can
introduce documentation without looking at it in the comment section of a Python program. So, in python of course, you can include comments
by writing this hash and then writing something after that.
  "video_url": "https://www.youtube.com/watch?v=G-XixYjFNnA&list=PLZ2ps__7DhBaDccbZRgiU1sHX2gZrQ-XT",
  "id": 3
},]
```

Milestone 4

h. Practice Assignment Question

Response:

```
[
  {
    "id": 1,
    "question": "This is a sample question",
    "correct_option": 1,
    "student_answer": 1,
    "week_id": 5,
    "options": [
      {
        "id": 1,
        "text": "option_1"
      },
      {
        "id": 2,
        "text": "option_2"
      },
      {
        "id": 3,
        "text": "option_3"
      },
      {
        "id": 4,
        "text": "option_4"
      }
    ]
  }
]
```

i. Coding Assignment Question

Response:

```
[
  {
    "id": 1,
    "question": "Write code to check if a number is prime or not.",
    "test_cases": [
      {
        "id": 1,
        "sample_input": "5",
        "sample_output": "6"
      }
    ]
  }
]
```


Milestone 4

j. Courses

Response:

```
[
  {
    "id": 4,
    "course_name": "PDSA",
    "user_id": 1
  },
  {
    "id": 1,
    "course_name": "DSA",
    "user_id": 1
  }
]
```

k. Create User

Request:

```
{
  "username": "vaibhav123",
  "role": "student",
  "email": "vaibhav123@gmail.com",
  "password": "securepassword"
}
```

Response:

```
{
  "message": "User created successfully",
  "user_id": 1
}
```

l. Mother Tongue Summarizer

m. Study Planner Agent

n. Support Assistant Agent

4. Code Implementation for APIs

a. Programming Mentor Agent

```
from fastapi import APIRouter, HTTPException
from app.agents.programming_mentor_agent import ProgrammingMentorAgent

router = APIRouter()
programming_mentor_agent = ProgrammingMentorAgent()
```

Milestone 4

```
@router.post("/programming-mentor")
def get_code_feedback(request: dict):
    """
    API Endpoint to analyze and optimize student solutions.
    Expects JSON input: { "coding_question": "text", "student_code": "text" }
    """
    try:
        coding_question = request.get("coding_question", "")
        student_code = request.get("student_code", "")

        if not coding_question or not student_code:
            raise HTTPException(status_code=400, detail="Coding question and student code are required.")

        feedback = programming_mentor_agent.evaluate_optimized_solution(coding_question, student_code)
        return {"optimization_feedback": feedback}

    except Exception as e:
        raise HTTPException(status_code=500, detail=str(e))
```

```
from langchain_openai import ChatOpenAI
from langchain.schema import HumanMessage
from app.core.config import config

class ProgrammingMentorAgent:
    def __init__(self):
        self.llm = ChatOpenAI(model="gpt-4o-mini", temperature=0.5, openai_api_key=config.OPENAI_API_KEY)

    def evaluate_optimized_solution(self, coding_question: str, student_code: str):
        """
        Analyze an already working solution to suggest improvements and optimizations.
        This function is only triggered after all test cases have passed.
        """
        prompt = (
            "You are an AI programming mentor. The student's code has passed all test cases. "
            "Provide only structured feedback and optimized code (if necessary) without extra commentary.\n\n"
            "### Coding Question:\n"
            f"{coding_question}\n\n"
            "### Student's Code:\n"
            f"{student_code}\n\n"
            "### Output Format:\n"
            "1. **Tips for Improvement:**\n"
            "- List only specific improvements in bullet points.\n"
            "- Keep explanations concise and technical.\n\n"
            "2. **Optimized Code (if necessary):**\n"
        )
```

Milestone 4

```
        "- If the student's solution can be optimized, provide an improved
version.\n"
        "- If no optimization is needed, return 'The solution is already
optimal.'\n\n"
        "### Example Response:\n"
        "**Tips for Improvement:**\n"
        "- Use a more efficient loop to reduce complexity.\n"
        "- Add docstrings to explain function behavior.\n\n"
        "**Optimized Code:**\n"
        "# Optimized code goes here\n"
    )

    response = self.llm([HumanMessage(content=prompt)])
    return response.content
```

b. Practice Evaluator Agent

```
from fastapi import APIRouter, HTTPException
from app.agents.practice_evaluator_agent import PracticeEvaluatorAgent

router = APIRouter()
practice_evaluator_agent = PracticeEvaluatorAgent()

@router.post("/evaluate-practice")
def evaluate_practice_submission(request: dict):
    """
    API Endpoint to evaluate multiple student practice assignments.
    Expects JSON input: { "submissions": [{ "assignment_question": "text",
"correct_answer": "text", "student_answer": "text" }] }
    """
    try:
        submissions = request.get("submissions", [])

        if not submissions or not isinstance(submissions, list):
            raise HTTPException(status_code=400, detail="A list of submissions
is required.")

        evaluation_results = []
        for submission in submissions:
            question = submission.get("assignment_question", "")
            correct_answer = submission.get("correct_answer", "")
            student_answer = submission.get("student_answer", "")

            if not question or not correct_answer or not student_answer:
                continue # Skip incomplete entries

            evaluation =
practice_evaluator_agent.evaluate_student_submission(question, correct_answer,
student_answer)
```

Milestone 4

```
        evaluation_results.append(evaluation)

    return {"evaluations": evaluation_results}

except Exception as e:
    raise HTTPException(status_code=500, detail=str(e))
```

```
from langchain_openai import ChatOpenAI
from langchain.schema import HumanMessage
from app.core.config import config
from app.agents.concept_identifier_agent import ConceptIdentifierAgent

class PracticeEvaluatorAgent:
    def __init__(self):
        self.llm = ChatOpenAI(model="gpt-4o-mini", temperature=0.5,
openai_api_key=config.OPENAI_API_KEY)
        self.concept_identifier = ConceptIdentifierAgent()

    def evaluate_student_submission(self, assignment_question: str,
correct_answer: str, student_answer: str):
        """
        Compare student's answer with the correct answer, identify mistakes,
        and generate extra practice questions on the same concept.
        """

        core_concept =
self.concept_identifier.identify_concept(assignment_question, correct_answer)

        prompt = (
            "You are an AI tutor analyzing a student's assignment
submission.\n"
            "Compare the student's answer with the correct answer and identify
mistakes.\n"
            "Generate your output in such a way that it feels you are talking
to the student\n"
            f"### Core Concept: {core_concept}\n\n"
            f"### Assignment Question: \n{assignment_question}\n\n"
            f"### Correct Answer: \n{correct_answer}\n\n"
            f"### Student's Answer: \n{student_answer}\n\n"
            "### Output Format:\n"
            "**Mistakes Identified:**\n"
            "- (List specific mistakes in bullet points)\n\n"
            "**Extra Practice Questions:**\n"
            "- (Generate 2 extra practice questions related to the core
concept)\n\n"
            "**Correct Answers for Practice Questions:**\n"
            "- (Provide correct answers for the extra questions)"
            "### Example Response:\n"
            "**Mistakes Identified:**\n"
            "- Your definition is vague and lacks clarity on function self-
calling.\n"
            "- No example is provided, making the explanation incomplete.\n\n"
            "**Extra Practice Questions:**\n"
```

Milestone 4

```
        "Extra question goes here"
        "**Correct Answers for Practice Questions:**\n"
        "Correct answer goes here\n"
    )

    response = self.llm([HumanMessage(content=prompt)])
    return {
        "assignment_question": assignment_question,
        "core_concept": core_concept,
        "feedback": response.content
    }
```

c. Conversation Agent

```
from fastapi import APIRouter, HTTPException
from app.agents.conversation_memory_agent import ConversationAgent

router = APIRouter()
conversation_agent = ConversationAgent()

@router.post("/conversation")
def handle_conversation(request: dict):
    """
    API Endpoint to manage user conversation.
    Expects JSON input: { "user_id": int, "message": "text" }
    """
    try:
        print("Request: ", request)
        user_id = request['user_id']
        message = request['message']

        if not user_id or not message:
            raise HTTPException(status_code=400, detail="User ID and message are required.")

        response = conversation_agent.process_message(user_id, message)
        return {"response": response}

    except Exception as e:
        raise HTTPException(status_code=500, detail=str(e))
```

d. Resource Recommender Agent

e. Login

```
@router.post("/verify-user")
def verify_user(request: VerifyUser, db: Session = Depends(get_db)):
    user = db.query(User).filter(User.email == request.email).first()
    if not user or not pwd_context.verify(request.password, user.password):
        raise HTTPException(status_code=401, detail="Invalid credentials")
    return {"message": "User verified", "user_id": user.id}
```

f. Create Lecture

Milestone 4

g.

```
@router.post("/create-lecture", response_model=LectureResponse)
def create_lecture(lecture: LectureCreate, db: Session = Depends(get_db)):
    new_lecture = Lecture(**lecture.dict())
    db.add(new_lecture)
    db.commit()
    db.refresh(new_lecture)
    return new_lecture
```

h. Fetch Lecture

```
@router.get("/get-lecture/week/{week_id}",
response_model=List[LectureResponse])
def get_lectures_by_week(week_id: int, db: Session = Depends(get_db)):
    lectures = db.query(Lecture).filter(Lecture.week_id == week_id).all()
    return lectures
```

i. Practice Assignment Question

```
@router.get("/mcq-questions/week/{week_id}",
response_model=List[MCQQuestionResponse])
def get_mcq_questions_by_week(week_id: int, db: Session = Depends(get_db)):
    questions = db.query(MCQQuestion).filter(MCQQuestion.week_id ==
week_id).all()
    if not questions:
        raise HTTPException(status_code=404, detail="No MCQ questions found for
this week")

    response = []
    for question in questions:
        options = db.query(MCQOption).filter(
            MCQOption.week_id == week_id,
            MCQOption.question_id == question.id
        ).all()

        response.append({
            "id": question.id,
            "question": question.question,
            "correct_option": question.correct_option,
            "student_answer": question.student_answer,
            "week_id": question.week_id,
            "options": [{"id": opt.id, "text": opt.option} for opt in options]
        })

    return response
```

j. Coding Assignment Question

```
@router.get("/coding-questions/{week_id}",
response_model=List[QuestionResponse])
def get_questions_with_test_cases(week_id: int, db: Session = Depends(get_db)):
    questions = (
        db.query(CodingQuestion)
        .filter(CodingQuestion.week_id == week_id)
        .all()
    )
```

Milestone 4

```
response_data = []
for question in questions:
    test_cases = (
        db.query(CodingTestCase)
        .filter(CodingTestCase.coding_id == question.id)
        .all()
    )

    response_data.append({
        "id": question.id,
        "question": question.question,
        "test_cases": [
            {"id": tc.id, "sample_input": tc.sample_input, "sample_output":
tc.sample_output}
            for tc in test_cases
        ]
    })

return response_data
```

k. Courses

```
@router.get("/courses/{user_id}", response_model=list[CourseResponseSchema])
def get_courses_by_user(user_id: int, db: Session = Depends(get_db)):
    courses = db.query(Course).filter(Course.user_id == user_id).all()
    if not courses:
        raise HTTPException(status_code=404, detail="No courses found for this
user.")
    return courses
```

l. Create User

```
@router.post("/create-user", response_model=dict)
def create_user(user: UserCreate, db: Session = Depends(get_db)):
    # Check if email or username already exists
    existing_user = db.query(User).filter((User.email == user.email) |
(User.username == user.username)).first()
    if existing_user:
        raise HTTPException(status_code=400, detail="User with this email or
username already exists")

    # Hash the password before storing it
    hashed_pwd = hash_password(user.password)

    new_user = User(
        username=user.username,
        role=user.role,
        email=user.email,
        password=hashed_pwd
    )
    db.add(new_user)
```

Milestone 4

```
db.commit()
db.refresh(new_user)

return {"message": "User created successfully", "user_id":
new_user.id}
```

- m. Mother Tongue Summarizer
- n. Study Planner Agent
- o. Support Assistant Agent
- p. Study Planner Agent
- q. User Details

```
@router.get("/user/{user_id}", response_model=UserResponse)
def get_user_by_id(user_id: int, db: Session = Depends(get_db)):
    user = db.query(User).filter(User.id == user_id).first()
    if not user:
        raise HTTPException(status_code=404, detail="User not found")
    return user
```

To be done:

1. Support Assistant Agent
2. Bug Reporting Endpoints

Sample Implementation:
Support Assistant Agent

```
@router.post("/support-assistant")
def get_code_feedback(request: dict):
    """
    API Endpoint to analyze codebase and identify bug from
    student
    reported bug
    """
    try:
        student_reported_issue = request.get("query", "")

        output =
support_assistant_agent.process_message({'query':
student_reported_issue})
        return {"result": output}

    except Exception as e:
        raise HTTPException(status_code=500, detail=str(e))
```

Bug Reporting Endpoint:

Milestone 4

```
@router.post("/report-bug", response_model= "placeholder")
def report_bug(db: Session = Depends(get_db)):
    pass

@router.get("/get-bugs", response_model="placeholder")
def get_bugs(db: Session = Depends(get_db)):
    pass

@router.get("/get-bug/{bug_id}",
response_model="placeholder")
def get_bug(db: Session = Depends(get_db)):
    pass
```

5. Running the APIs

Start FastAPI Server

`uvicorn app.main:app --reload`

TEST IN POSTMAN

Method: POST

URL : <http://127.0.0.1:8000/api/programming-mentor>

Body:

```
{
  "coding_question": "Write a Python function to check if a number is prime."
  "student_code": "def is_prime(n): return all(n % i != 0 for i in range(2, n))"
}
```

```
{
  "optimization_feedback": "**Tips for Improvement:**\n- Handle edge cases for numbers less than 2.\n- Limit the loop to check divisibility up to the square root of `n` for efficiency.\n- Add docstrings to explain function behavior.\n\n**Optimized Code:**\n```\npython\nimport math\n\ndef is_prime(n):\n    \"\"\"Check if a number is prime.\"\"\"\n    if n < 2:\n        return False\n    for i in range(2, int(math.sqrt(n)) + 1):\n        if n % i == 0:\n            return False\n    return True\n```\n}\n}
```

Milestone 5

API Test:

1. Coding-questions (GET /coding-questions/{week_id})

a. Valid case:

Inputs - week_id=1

Expected Output:

The API should return two coding questions along with their respective test cases.

- I. "What is recursion?" with a test case (sample_input="5", sample_output="120").
- II. "Explain dynamic programming." with a test case (sample_input="4", sample_output="24").

Actual Output : The API returned two coding questions with the correct test cases matching the expected values.

Result: Success- API returned correct data for week_id=1.

```
def test_get_coding_questions(mock_db, monkeypatch):
    """Test the GET /coding-questions/{week_id} API endpoint."""

    # Sample data
    mock_questions = [
        CodingQuestion(id=1, question="What is recursion?", week_id=1),
        CodingQuestion(id=2, question="Explain dynamic programming.", week_id=1),
    ]

    mock_test_cases = {
        1: [CodingTestCase(id=101, coding_id=1, sample_input="5", sample_output="120")],
        2: [CodingTestCase(id=102, coding_id=2, sample_input="4", sample_output="24")]
    }

    # Mock database queries
    def mock_query(model):
        mock_query_obj = MagicMock()
        if model == CodingQuestion:
            mock_query_obj.filter.return_value.all.return_value = mock_questions
        elif model == CodingTestCase (parameter) coding_id: Any
            def filter_by_coding_id(coding_id):
                return mock_test_cases.get(coding_id, [])
            mock_query_obj.filter.side_effect = filter_by_coding_id
        return mock_query_obj

    mock_db.query.side_effect = mock_query

    mock_db.query.side_effect = mock_query

    # Monkeypatch FastAPI dependency override
    monkeypatch.setattr("app.api.routes.coding_questions.get_db", lambda: mock_db)

    # Make request
    response = client.get("/coding-questions/1")

    # Assertions
    assert response.status_code == 200
    response_data = response.json()

    assert len(response_data) == 2
    assert response_data[0]["id"] == 1
    assert response_data[0]["question"] == "What is recursion?"
    assert len(response_data[0]["test_cases"]) == 1
    assert response_data[0]["test_cases"][0]["sample_input"] == "5"
    assert response_data[0]["test_cases"][0]["sample_output"] == "120"
```

b. Invalid case:

Inputs: week_id=99

Milestone 5

Expected Output:

For week_id=99: The API should return a 404 Not Found error with a message "No coding questions found for this week."

Actual Output:

For week_id=99: The API returned a 404 response with the expected error message.

Result: Success - API correctly returned 404 Not Found for week_id=99, indicating no data was available.

```
1 def test_get_coding_questions_not_found(mock_db, monkeypatch):
2     """Test the GET /coding-questions/{week_id} endpoint when no questions are found."""
3
4     # Mock database queries returning no questions
5     def mock_query(model):
6         mock_query_obj = MagicMock()
7         mock_query_obj.filter.return_value.all.return_value = []
8         return mock_query_obj
9
10    mock_db.query.side_effect = mock_query
11
12    # Monkeypatch FastAPI dependency override
13    monkeypatch.setattr("app.api.routes.coding_questions.get_db", lambda: mock_db)
14    mock_db.query.side_effect = mock_query
15
16    # Monkeypatch FastAPI dependency override
17    monkeypatch.setattr("app.api.routes.coding_questions.get_db", lambda: mock_db)
18
19    # Make request
20    response = client.get("/coding-questions/99") # Using a week_id that has no questions
21
22    # Assertions
23    assert response.status_code == 404
24    response_data = response.json()
25    assert "detail" in response_data
26    assert response_data["detail"] == "No coding questions found for this week."
```

2. POST/create-lecture:

a. Valid case (POST/create-lecture/{week_id}):

Inputs:

```
{
  "week_id": 2,
  "lecture_name": "Advanced Python",
  "transcript_content": "Advanced content.",
  "video_url": "https://example.com/advanced"
}
```

Expected Output: The API should return a **200 OK** response with the created lecture details:

```
{
  "id": 1,
  "week_id": 2,
  "lecture_name": "Advanced Python",
  "transcript_content": "Advanced content.",
  "video_url": "https://example.com/advanced"
}
```

Actual Output: The API returned **200 OK** with the correct lecture details matching expected values.

Result: API correctly created a lecture with valid input.

Milestone 5

```
17 def test_create_lecture(mock_db, monkeypatch):
18     """Test the POST /create-lecture API endpoint."""
19     mock_lecture = Lecture(id=1, week_id=2, lecture_name="Advanced Python",
20
21     mock_db.add.return_value = None
22     mock_db.commit.return_value = None
23     mock_db.refresh.return_value = mock_lecture
24     monkeypatch.setattr("app.core.database.get_db", lambda: mock_db)
25
26     response = client.post(
27         "/create-lecture",
28         json={"week_id": 2, "lecture_name": "Advanced Python", "transcript_
29     )
30
31     assert response.status_code == 200
32     data = response.json()
33     assert data["lecture_name"] == "Advanced Python"
34     assert data["week_id"] == 2
35     assert data["video_url"] == "https://example.com/advanced"
```

b. Invalid case (POST/create-lecture):

Inputs : {}

Expected Output: The API should return a **422 Unprocessable Entity** error due to missing required fields.

Actual Output: The API returned a **422** error as expected.

Result: Success - API correctly rejected the request due to missing data.

```
def test_create_lecture_missing_data():
    """Test creating a lecture with missing required fields."""
    response = client.post("/create-lecture", json={})
    assert response.status_code == 422 # Unprocessable entity
```

3. Get-lectures:

a. Valid case(GET/get-lecture/{lecture id}):

Inputs: Valid lecture_id (1)

Expected Output: The API should return a **200 OK** response with lecture details:

```
{
  "id": 1,
  "week_id": 1,
  "lecture_name": "Intro to Python",
  "transcript_content": "Basic content.",
  "video_url": "https://example.com/intro"
}
```

Actual Output: The API returned **200 OK** with the correct lecture details.

Result: Success - API correctly retrieved lecture for lecture_id=1.

```
def test_get_lecture(mock_db, monkeypatch):
    """Test the GET /get-lecture/{lecture_id} API endpoint."""
    mock_db.query.return_value.filter.return_value.first.return_value =
    Lecture(id=1, week_id=1, lecture_name="Intro to Python",
    transcript_content="Basic content.", video_url="https://example.com/
    intro")

    monkeypatch.setattr("app.core.database.get_db", lambda: mock_db)
    response = client.get("/get-lecture/1")

    assert response.status_code == 200
    data = response.json()
    assert data["id"] == 1
    assert data["lecture_name"] == "Intro to Python"
```

b. valid case(GET/get-lecture)

Inputs: No input parameters required.

Expected Output: The API should return a 200 OK response with a list of lectures.

```
[
  {
    "id": 1,
```

Milestone 5

```
"week_id": 1,
"lecture_name": "Intro to Python",
"transcript_content": "Basic content.",
"video_url": "https://example.com/intro"
}
]
```

Actual Output: The API returned 200 OK with a list containing at least one lecture.
Result: Success- API correctly retrieved all lectures.

```
def test_get_all_lectures(mock_db, monkeypatch):
    """Test the GET /get-lectures API endpoint."""
    mock_db.query.return_value.all.return_value = [
        Lecture(id=1, week_id=1, lecture_name="Intro to Python", transcript
    ]

    monkeypatch.setattr("app.core.database.get_db", lambda: mock_db)
    response = client.get("/get-lectures")

    assert response.status_code == 200
    data = response.json()
    assert isinstance(data, list)
    assert len(data) > 0
    assert data[0]["lecture_name"] == "Intro to Python"
```

c. invalid case(GET/get-lecture/{lecture_id})

Inputs: Non-existent lecture_id (999)

Expected Output: The API should return a **404 Not Found** error with message "**Lecture not found.**"

Actual Output: The API returned **404 Not Found** with the expected error message.

Result: Success - API correctly returned 404 for a non-existent lecture.

```
def test_get_lecture_not_found(mock_db, monkeypatch):
    """Test retrieving a lecture that does not exist."""
    mock_db.query.return_value.filter.return_value.first.return_value =
    None

    monkeypatch.setattr("app.core.database.get_db", lambda: mock_db)
    response = client.get("/get-lecture/999")

    assert response.status_code == 404
    data = response.json()
    assert data["detail"] == "Lecture not found"
```

d. valid case(GET/get-lecture/week/{week_id})

Inputs: Valid week_id (1)

Expected Output: The API should return a **200 OK** response with lectures for that week:

```
[
{
    "id": 1,
    "week_id": 1,
    "lecture_name": "Intro to Python",
    "transcript_content": "Basic content.",
    "video_url": "https://example.com/intro"
}
]
```

Actual Output: The API returned **200 OK** with the expected lecture details.

Result: Success- API correctly returned an empty list for week_id=999.

Milestone 5

```
def test_get_lectures_by_week(mock_db, monkeypatch):
    """Test the GET /get-lecture/week/{week_id} API endpoint."""
    mock_db.query.return_value.filter.return_value.all.return_value = [
        Lecture(id=1, week_id=1, lecture_name="Intro to Python",
            transcript_content="Basic content.", video_url="https://example.com/intro")
    ]

    monkeypatch.setattr("app.core.database.get_db", lambda: mock_db)
    response = client.get("/get-lecture/week/1")

    assert response.status_code == 200
    data = response.json()
    assert isinstance(data, list)
    assert len(data) > 0
    assert data[0]["week_id"] == 1
```

e. valid case(GET/get-lecture/week/{week_id})

Inputs: Non-existent week_id (999)

Expected Output: The API should return a 200 OK response with an empty list ([]), indicating no lectures for this week.

Actual Output: The API returned 200 OK with an empty list as expected.

Result: API correctly returned an empty list for week_id=999.

```
def test_get_lectures_by_week_empty(mock_db, monkeypatch):
    """Test retrieving lectures for a week with no lectures."""
    mock_db.query.return_value.filter.return_value.all.return_value = []

    monkeypatch.setattr("app.core.database.get_db", lambda: mock_db)
    response = client.get("/get-lecture/week/999")

    assert response.status_code == 200
    data = response.json()
    assert isinstance(data, list)
    assert len(data) == 0
```

4. MCQ QUESTIONS:

a. Valid case(GET/mcq-questions/week/{week_id}):

Inputs: Valid Week Id – 1

Expected Output: The API should return a 200 OK response with MCQ questions and their respective options

```
[
  {
    "id": 1,
    "question": "What is Python?",
    "correct_option": 2,
    "student_answer": null,
    "week_id": 1,
    "options": [
      {
        "id": 1,
        "week_id": 1,
        "question_id": 1,
        "text": "A programming language"
      },
      {
        "id": 2,
        "week_id": 1,
        "question_id": 1,
        "text": "A snake"
      }
    ]
  }
]
```


Milestone 5

Actual Output: The API returned **200 OK** with the expected MCQ question details.

Result: Success- API correctly retrieved MCQ questions for week_id=1.

```
def test_get_mcq_questions_by_week(mock_db, monkeypatch):
    """Test the GET /mcq-questions/week/{week_id} endpoint."""
    mock_question = MCQQuestion(id=1, question="What is Python?",
    correct_option=2, student_answer=None, week_id=1)
    mock_option1 = MCQOption(id=1, week_id=1, question_id=1, option="A
    programming language")
    mock_option2 = MCQOption(id=2, week_id=1, question_id=1, option="A
    snake")

    mock_db.query.return_value.filter.return_value.all.side_effect =
    [[mock_question], [mock_option1, mock_option2]]
    monkeypatch.setattr("app.api.routes.mcq_routes.get_db", lambda:
    mock_db)

    response = client.get("/mcq-questions/week/1")

    assert response.status_code == 200
    data = response.json()
    assert isinstance(data, list)
    assert len(data) == 1
    assert data[0]["question"] == "What is Python?"
    assert data[0]["correct_option"] == 2
    assert len(data[0]["options"]) == 2
    assert data[0]["options"][0]["text"] == "A programming language"
```

b. Invalid case(GET/mcq-questions/week/{week_id}):

Inputs: Non-existent week_id (99)

Expected Output: The API should return a 404 Not Found error with message "No MCQ questions found for this week."

Actual Output: The API returned 404 Not Found with the expected error message.

Result: Success- API correctly returned 404 for week_id=99, indicating no MCQ questions were found.

```
def test_get_mcq_questions_by_week_not_found(mock_db, monkeypatch):
    """Test the GET /mcq-questions/week/{week_id} endpoint when no
    questions exist."""
    mock_db.query.return_value.filter.return_value.all.return_value = []
    monkeypatch.setattr("app.api.routes.mcq_routes.get_db", lambda:
    mock_db)

    response = client.get("/mcq-questions/week/99")

    assert response.status_code == 404
    assert response.json()["detail"] == "No MCQ questions found for this
    week"
```

5. SUPPORT ASSISTANT:

a. Valid case(POST/support-assistant):

Inputs: {}

Expected Output: The API should return a 200 OK response, assuming the system handles empty input gracefully.

```
{
  "result": "<some default or empty response>"
}
```

Actual Output: The API returned 200 OK, providing a response even when input was missing.

Result: Success: API handled missing input without failure.

Milestone 5

```
def test_get_support_feedback_missing_data():
    """Test /support-assistant endpoint with missing input data."""
    response = client.post("/support-assistant", json={})
    assert response.status_code == 200 # Assuming empty query still
    returns a response
    data = response.json()
    assert "result" in data
```

b. Invalid case(POST/support-assistant):

Inputs: Internal server error simulation

Expected Output: The API should return a 500 Internal Server Error response with a message

Actual Output: The API returned 500 Internal Server Error with the expected error message.

Result: Success- API correctly returned a 500 error for internal failure scenarios.

```
def test_get_support_feedback_internal_error(monkeypatch):
    """Test /support-assistant endpoint handling internal server errors.
    """
    def mock_process_message(*args, **kwargs):
        raise Exception("Unexpected error")

    monkeypatch.setattr("app.api.routes.support_assistant_routes.
        support_assistant_agent.process_message", mock_process_message)

    response = client.post("/support-assistant", json={"query": "My loop
        runs infinitely."})
    assert response.status_code == 500
    data = response.json()
    assert "detail" in data
    assert data["detail"] == "Unexpected error"
```

6. Conversation:

a. Valid case(POST/conversation)

Inputs:

```
{"user_id": 1,
"message": "Hello!"}
```

Expected Output:

```
{"response": "Mocked response"}
```

Actual Output : The API returned 200 OK with the expected response.

Result: Success - API correctly processed valid input and returned expected response.

b. invalid case(POST/conversation)

Inputs:

```
{
  "user_id": 1
}
```

Expected Output: The API should return a 400 Bad Request response.

Actual Output: The API returned 400 Bad Request, correctly handling incomplete input.

Result: Success - API correctly returned 400 for missing required fields.

Milestone 5

```
def test_handle_conversation_missing_fields():
    """Test /conversation endpoint with missing fields."""
    response = client.post("/conversation", json={"user_id": 1})
    assert response.status_code == 400

    response = client.post("/conversation", json={"message": "Hello!"})
    assert response.status_code == 400
```

c. invalid case(POST/conversation)

Inputs:

```
{"user_id": 1,
 "message": "Hello!"}
```

Expected Output:

```
{"detail": "Unexpected error"}
```

Actual Output : The API returned 500 Internal Server Error with the expected error message.

Result: API correctly returned 500 for an internal error.

7. RESOURCE RECOMENDER

a. Valid case(POST/resource-recommender)

Inputs:

```
{"query": "Explain Binary Search Trees with examples."}
```

Expected Output: The API should return a 200 OK response with the recommended resource details

```
{
  "result": {
    "book": "Introduction to Algorithms",
    "page_number": 102,
    "topic": "Binary Search Trees"
  }
}
```

Actual Output: The API returned 200 OK with the expected recommended resource details.

Result: Success - API correctly returned recommended resource details for a valid query.

```
def test_get_code_feedback(mock_resource_recommender_agent):
    """Test the /resource-recommender endpoint with a DSA-related query.
    """
    response = client.post("/resource-recommender", json={"query":
        "Explain Binary Search Trees with examples."})

    assert response.status_code == 200
    data = response.json()
    assert "book" in data["result"]
    assert "page_number" in data["result"]
    assert "topic" in data["result"]
    assert data["result"]["book"] == "Introduction to Algorithms"
    assert data["result"]["page_number"] == 102
    assert data["result"]["topic"] == "Binary Search Trees"

    mock_resource_recommender_agent.
        retrieve_recommended_resource_from_query.assert_called_once_with
        ("Explain Binary Search Trees with examples.")
```

b. invalid case(POST/resource-recommender)

Inputs:

```
{"query": "Explain Binary Search Trees with examples."}
```

Expected Output:

```
{"detail": "Unexpected error"}
```

Actual Output: The API returned 500 Internal Server Error with the expected error message.

Milestone 5

Result: Success- API correctly returned 500 for an internal error.

```
def test_get_code_feedback_internal_error(monkeypatch):
    """Test /resource-recommender endpoint handling internal server
    errors."""
    def mock_retrieve_recommended_resource_from_query(*args, **kwargs):
        raise Exception("Unexpected error")

    monkeypatch.setattr("app.api.routes.resource_recommender_routes.
    resource_recommender_agent.retrieve_recommended_resource_from_query",
    mock_retrieve_recommended_resource_from_query)

    response = client.post("/resource-recommender", json={"query":
    "Explain Binary Search Trees with examples."})
    assert response.status_code == 500
    data = response.json()
    assert "detail" in data
    assert data["detail"] == "Unexpected error"
```

8. PROGRAMING MENTROR

a. valid case(POST/programming-mentor)

Input:

```
{
  "coding_question": "Implement Fibonacci sequence using recursion.",
  "student_code": "def fib(n): return 1 if n<=2 else fib(n-1) + fib(n-2)"
}
```

Expected Output:

```
{
  "optimization_feedback": {
    "feedback": "Your solution is correct but can be optimized using dynamic programming.",
    "suggested_improvement": "Consider using memoization to reduce redundant computations."
  }
}
```

Actual Output: The API returned 200 OK with the expected feedback and suggested improvement.

Result: The API correctly evaluated the student's code and provided optimization feedback.

```
def test_get_code_feedback(mock_programming_mentor_agent):
    """Test the /programming-mentor endpoint with a coding question and
    student code."""
    response = client.post("/programming-mentor", json={
        "coding_question": "Implement Fibonacci sequence using recursion.",
        "student_code": "def fib(n): return 1 if n≤2 else fib(n-1) + fib
        (n-2)"
    })

    assert response.status_code == 200
    data = response.json()
    assert "optimization_feedback" in data
    assert "feedback" in data["optimization_feedback"]
    assert "suggested_improvement" in data["optimization_feedback"]
    assert data["optimization_feedback"]["feedback"] == "Your solution is
    correct but can be optimized using dynamic programming."
    assert data["optimization_feedback"]["suggested_improvement"] ==
    "Consider using memoization to reduce redundant computations."

    mock_programming_mentor_agent.evaluate_optimized_solution.
    assert_called_once_with(
        "Implement Fibonacci sequence using recursion.",
        "def fib(n): return 1 if n≤2 else fib(n-1) + fib(n-2)"
    )
```

b. invalid case(POST/programming-mentor)

Inputs: {}

Expected Output:

```
{ "detail": "Coding question and student code are required." }
```

Actual Output: The API returned 400 Bad Request with the expected error message.

Milestone 5

Result: Success- The API correctly returned a 400 error for missing data

```
def test_get_code_feedback_missing_data():
    """Test /programming-mentor endpoint with missing input data."""
    response = client.post("/programming-mentor", json={})
    assert response.status_code == 400
    data = response.json()
    assert "detail" in data
    assert data["detail"] == "Coding question and student code are required."
```

c. invalid case(POST/programming-mentor)

Inputs:

```
{
  "coding_question": "Implement Fibonacci sequence using recursion.",
  "student_code": "def fib(n): return 1 if n<=2 else fib(n-1) + fib(n-2)"
}
```

Expected Output:

```
{"detail": "Unexpected error"}
```

Actual Output: The API returned 500 Internal Server Error with the expected error message.

Result: The API returned 500 Internal Server Error with the expected error message.

```
def test_get_code_feedback_internal_error(monkeypatch):
    """Test /programming-mentor endpoint handling internal server errors.
    """
    def mock_evaluate_optimized_solution(*args, **kwargs):
        raise Exception("Unexpected error")

    monkeypatch.setattr("app.api.routes.programming_mentor_routes.
        programming_mentor_agent.evaluate_optimized_solution",
        mock_evaluate_optimized_solution)

    response = client.post("/programming-mentor", json={
        "coding_question": "Implement Fibonacci sequence using recursion.",
        "student_code": "def fib(n): return 1 if n≤2 else fib(n-1) + fib(n-2)"
    })
    assert response.status_code == 500
    data = response.json()

    response = client.post("/programming-mentor", json={
        "coding_question": "Implement Fibonacci sequence using recursion.",
        "student_code": "def fib(n): return 1 if n≤2 else fib(n-1) + fib(n-2)"
    })
    assert response.status_code == 500
    data = response.json()
    assert "detail" in data
    assert data["detail"] == "Unexpected error"
```

9. MTT SUMMARIZER

a. Valid case(POST/mtt-summarizer)

Inputs:

```
{
  "lecture_transcript": "This is a test transcript.",
  "mother_tongue": "Spanish"
}
```

Expected Output: The API should return a **200 OK** response with the translated transcript:

```
{"translated_transcript": "Mocked translated transcript"}
```

Milestone 5

Actual Output: The API returned **200 OK** with the expected translated transcript.

```
def test_get_mtt_summary(mock_mtt_summarizer_agent):
    """Test the /mtt-summarizer endpoint."""
    response = client.post("/mtt-summarizer", json={"lecture_transcript":
    "This is a test transcript.", "mother_tongue": "Spanish"})

    assert response.status_code == 200
    data = response.json()
    assert "translated_transcript" in data
    assert data["translated_transcript"] == "Mocked translated transcript"

    mock_mtt_summarizer_agent.retrieve_translated_transcript.
    assert_called_once_with("This is a test transcript.", "Spanish")
```

b. invalid case(POST/mtt-summarizer)

Input:

```
{
  "lecture_transcript": "Sample text.",
  "mother_tongue": "French"
}
```

Expected Output:

```
{ "detail": "Unexpected error" }
```

Actual Output: The API returned **500 Internal Server Error** with the expected error message

Result: The API correctly returned a **500 error** for internal server errors.

```
def test_get_mtt_summary(mock_mtt_summarizer_agent):
    """Test the /mtt-summarizer endpoint."""
    response = client.post("/mtt-summarizer", json={"lecture_transcript":
    "This is a test transcript.", "mother_tongue": "Spanish"})

    assert response.status_code == 200
    data = response.json()
    assert "translated_transcript" in data
    assert data["translated_transcript"] == "Mocked translated transcript"

    mock_mtt_summarizer_agent.retrieve_translated_transcript.
    assert_called_once_with("This is a test transcript.", "Spanish")
```

c. invalid case(POST/mtt-summarizer)

Inputs:

```
{s
  "lecture_transcript": "Sample text.",
  "mother_tongue": "French"
}
```

Expected Output:

```
{
  "detail": "Unexpected error"
}
```

Actual Output: The API returned **500 Internal Server Error** with the expected error message.

Result: The API correctly returned a **500 error** for internal server errors.

Milestone 5

```
def test_get_mtt_summary_internal_error(monkeypatch):
    """Test /mtt-summarizer endpoint handling internal server errors."""
    def mock_retrieve_translated_transcript(*args, **kwargs):
        raise Exception("Unexpected error")

    monkeypatch.setattr("app.api.routes.programming_mentor_routes.
mtt_summarizer_agent.retrieve_translated_transcript",
mock_retrieve_translated_transcript)

    response = client.post("/mtt-summarizer", json={"lecture_transcript":
"Sample text.", "mother_tongue": "French"})
    assert response.status_code == 500
    data = response.json()
    assert "detail" in data
    assert data["detail"] == "Unexpected error"
```

Milestone 6

Instructions to run your application

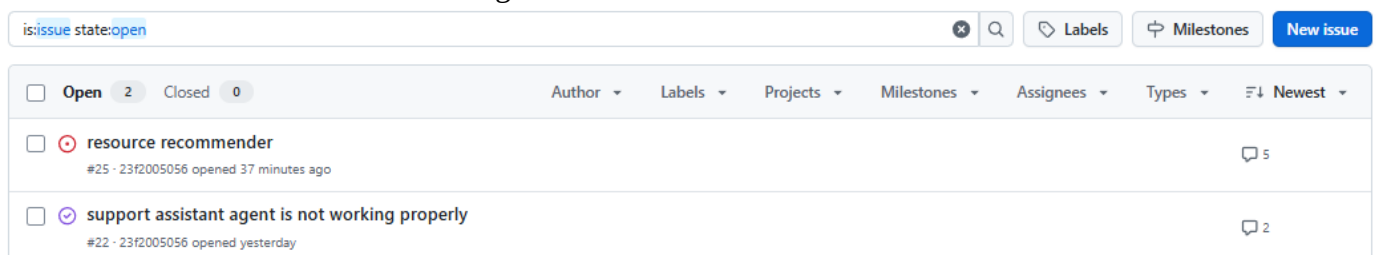
- use Docker
- Install Python, if not already installed.
- Install NodeJs, if not already installed.
- Open Terminal and navigate to the project directory, and then to the backend folder..
- Create a virtual environment by running the following command: ***python3 -m venv env_name***
- Activate the virtual environment: ***source env_name/bin/activate***
- Install dependencies: ***pip install -r requirements.txt***
- Run the flask api - ***uvicorn app.main:app --reload***
- Open a new Terminal tab. Navigate to the frontend directory: ***cd ../frontend***
- Setup the project: ***npm install***
- Compile the frontend: ***npm run serve***
- Access the frontend - ***http://127.0.0.1:8000***

Code Review, Issue Reporting and Tracking

For daily communication, WhatsApp was used including raising issues, obtaining approvals and basic collaboration. GitHub is used for code and documentation management. Each team member has a separate branch, while the **integration** branch is used to merge and validate all work against requirements. Changes are committed and pushed as needed.

Issues arising from peer code reviews and testing were logged and assigned to the developer responsible. Upon fixing the issue, the tester would retest and close the issue. A few screenshots are given below.

Screenshots of issues status and tracking of issues:



Milestone 6

support assistant agent is not working properly #22

Open



23f2005056 opened 12 minutes ago

...

The Support Assistant Agent is integrated with the API but only shows possible causes without displaying the suggested code block.

Create sub-issue



21f2001228 now

...

Api agent now also showing suggested code block.



23f2005056 now

Author ...

@21f2001228



resource recommender #25

Open



23f2005056 opened 38 minutes ago

...

chat does not support resource recommendation
[@priyanshujiiii](#)

Create sub-issue



priyanshujiiii 32 minutes ago

...

its under process currently i am resolving with frontend issue
i am integrating resource recommendation



priyanshujiiii 26 minutes ago

...

resource recommender is integrated



23f2005056 26 minutes ago

Author ...

the resource recommender is showing the recommended books names only
but its not showing the links to those books, please make them as links
[@priyanshujiiii](#)



priyanshujiiii 25 minutes ago

...

i am fixing this



priyanshujiiii 24 minutes ago

...

now its working fine

Milestone 6

Screenshots of few commits performed by team members on github

completed practice evaluator agent: #2

Merged 21f1000096 merged 2 commits into `development` from `feature_pa_eval_agent_09022025` last month

Conversation 3 Commits 2 Checks 0 Files changed 5

vaibhavgupta-iitm commented on Feb 19

Completed practice evaluator agents, changed gpt model to be used for lesser resource consumption.

21f1000096 completed practice evaluator agent: 2d57fba

vaibhavgupta-iitm commented on Feb 19 **Author**

@21f1000096 Please review the merge.

21f1000096 commented last month

Please look into code cleanups as suggested in our meeting.

TECHNOLOGIES AND TOOLS USED:

Library / Framework / Language	Usage
Python	Core Programming
VUE	Frontend
Flask	Backend
SQLAlchemy	Database
werkzeug.security	Password hashing
Pytest	Python tool for testing
Github	Code Review , Issue reporting and Issue tracking
Jira	Project management and scheduling
Open API	GEN AI
Flask CORS	Communication between HTTP methods and Flask
JWT	Authentication
Swagger	API yaml file creation

Milestone 6

API endpoints:

Since milestones 4 and 5, numerous API endpoints have been added—some to enhance the project and others to bring it to completion. List of total api end points are given below:

- POST /api/programming-mentor - Get Code Feedback
- POST /api/evaluate-practice - Evaluate Practice Submission
- POST /api/resource-recommender - Get Code Feedback
- POST /api/file-selector - Select Relevant Files
- POST /api/student_responses/{user_id}/{week_id} - Create User Responses
- GET /api/responses/{week_id} - Get Responses By Week
- GET /api/mcq-questions/week/{week_id} - Get MCQ Questions By Week
- GET /api/coding-questions/{week_id} - Get Coding Questions With Test Cases
- POST /api/conversation - Handle Conversation
- POST /api/topics - Create Topic
- GET /api/topics/{user_id} - Get Topics
- GET /api/conversations/{user_id}/{topic_id} - Get Conversations By User And Topic
- POST /api/create-support-ticket - Create Support Ticket
- POST /api/bug-fix-assistant - Assist With Bug Fix
- GET /api/languages - Get Available Languages
- POST /api/mtt-summarizer - Translate Transcript
- GET /api/transcript-checker - Health Check
- GET / - Root
- POST /api/create-user - Create User
- POST /api/verify-user - Verify User
- POST /api/create-lecture - Create Lecture
- GET /api/get-lectures - Get All Lectures
- GET /api/get-lecture/{lecture_id} - Get Lecture
- GET /api/get-lecture/week/{week_id} - Get Lectures By Week
- GET /api/courses - Get All Courses
- GET /api/courses/{user_id} - Get Courses By User
- GET /api/courses/{course_id} - Get Course By ID